하이킨아시 반전 패턴 검증 보고서

원래 규칙에서 출발해 손절폭 · 시장 필터 · 손익비를 바꿔가며 끝까지 확인한 기록

한국 주식 2,745 종목(상장폐지 117 포함) · 2021 년 1 월 이후 일봉 · 작성일 2026-10-16

한 줄 결론. 이 규칙은 매매 신호로 사용하기에는 부적절하다. 거래비용 반영 시에, 초기에 가정한 어떤 조합에서도 평균수익이 마이너스였고, 청산 규칙을 전부 뺀 테스트에서도 같은 종목의 무작위 진입보다 나은 점이 없었다.

요약

- **질문:** 하이킨아시 도지 반전 + EMA20 돌파 신호가 비용을 반영한 뒤에도 랜덤 진입보다 나은가?
- **방법:** 생존편향 완화, 다음날 시가 진입, 거래비용 0.3%, 유동성 필터, 2024 년 기준 구간 분리, 랜덤 대조군.
- **과정:** 원래 규칙 → 표본 확대 → 손절폭 2 배 → 코스피 필터 → 손익비 변경 → 청산 규칙 제거, 순서로 하나씩 막힌 길을 확인했다.
- **결과:** 72 개 조합 전부 평균수익이 마이너스 (−0.27% ~ −1.70%). 고정 보유기간 27 개 조합 중 25 개에서 랜덤보다 나쁨.
- **의미:** 전략이 돈이 된다는 주장이 아니라, 검증을 끝까지 해서 아니라는 것을 확인한 프로젝트다.

---

1. 연구 질문과 규칙

투자대회용으로 하이킨아시 스캐너를 만들었고, 이후 "이 규칙이 우연이 아닌 우위를 가지는가"를 검증하는 쪽으로 목표를 바꿨다. 스캐너는 후보를 추리는 도구이고 자동매매가 아니다. 검증 대상은 아래 네 가지 필수 조건을 모두 만족하는 신호다.

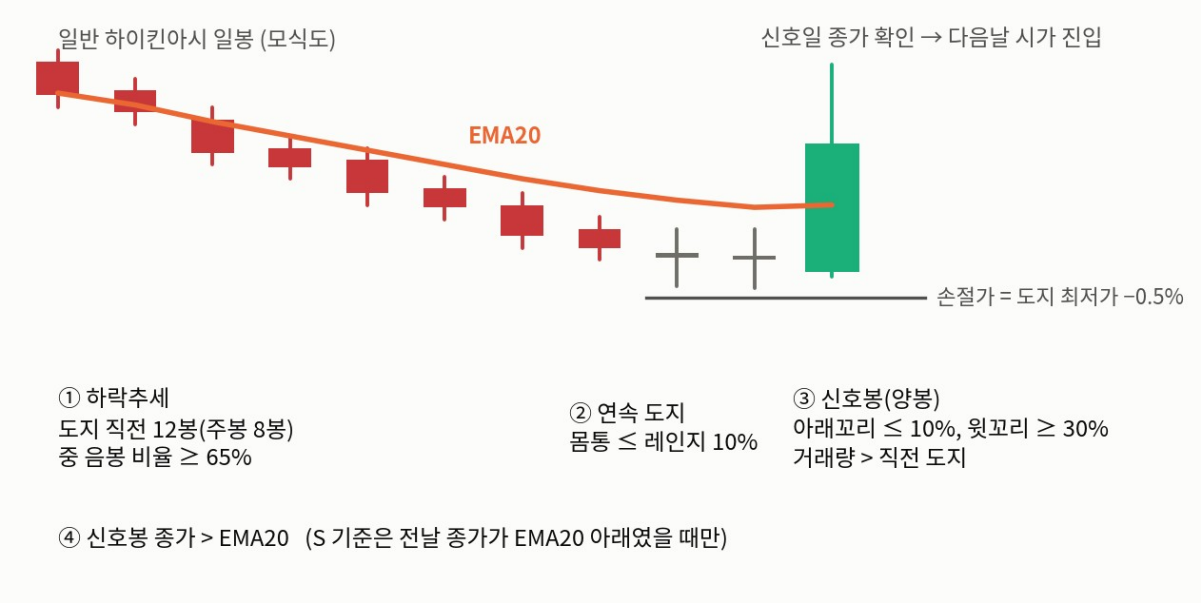


그림 1. 신호 규칙 모식도. EMA20 조건은 세 가지로 나눠 비교했다: N(조건 없음, 기준선), R(신호봉 종가 > EMA20), S(전날 종가는 EMA20 아래, 신호봉 종가는 위).

청산: 손절가(도지 최저가 −0.5%), 익절가(진입가 + 손익비 × 손절폭), 또는 스무스 하이킨아시 양봉→음봉 전환 시 다음날 시가 매도 중 먼저 오는 것. 최대 60 거래일.

2. 검증 설계

백테스트가 실제보다 좋아 보이는 흔한 이유를 먼저 막았다.

위험	처리
생존편향	2021 년 이후 상장폐지·합병 종목 117 개(스팩 제외)를 가격 데이터와 함께 포함. 전체 2,745 종목.
룩어헤드	신호일 종가를 확인한 뒤 다음날 시가에 진입.
현실적 체결	왕복 거래비용 0.3%. 시가가 이미 손절가 아래 / 익절가 위면 시가 체결. 같은 날 둘 다 닿으면 손절 우선.
유동성	5 일 평균 거래대금 30 억 이상인 신호만 평가(대회 규정). 전체 신호의 약 26%만 통과.
과최적화	2024-01-01 전후로 구간을 나눠 같은 방향인지 확인. 검증 구간을 보고 파라미터를 다시 조정하지 않음.
우연	같은 종목·같은 손절폭·손익비·같은 코스피 국면의 무작위 진입을 대조군으로 두고 p 값과 t 값을 계산.
다중비교	72 개 조합을 비교하므로 $p \le 0.05$ 가 우연만으로도 약 3.6 개 나온다고 처음부터 명시.

3. 연구 기록: 무엇을 시도했고 무엇이 막혔나

① 원래 규칙으로 무작위 300 종목 테스트

규칙을 네 가지 필수 조건으로 정리한 뒤 먼저 무작위 300 종목(상장폐지 10 개 포함)으로 테스트를 시행했다. 유동성을 통과한 거래 기준으로 N −0.38%, R −0.61%, S −0.02%(PF 1.00)였다.

② 전체 2,745 종목으로 확대

거래가 200 건대인 표본은 평균수익의 오차가 크기 때문에 전체로 늘려 확인했다. S 는 −0.02%에서 **−0.67%**(1,874 건, PF 0.89)로 내려갔고 p 값은 0.14 가 됐다. 세 기준 모두 랜덤 진입과 통계적 유의미함을 발견하기는 어려웠다. (p 0.14~0.45).

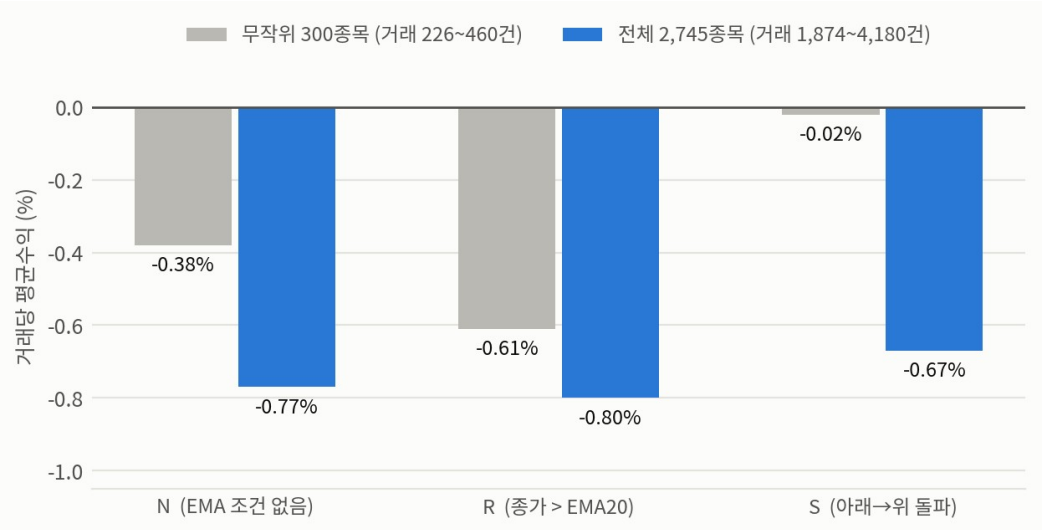


그림 2. 표본을 300 종목에서 전체로 늘리자 0%처럼 보이던 S 가 −0.67%로 내려갔다. 작은 표본의 좋은 결과는 의심해야 한다는 예.

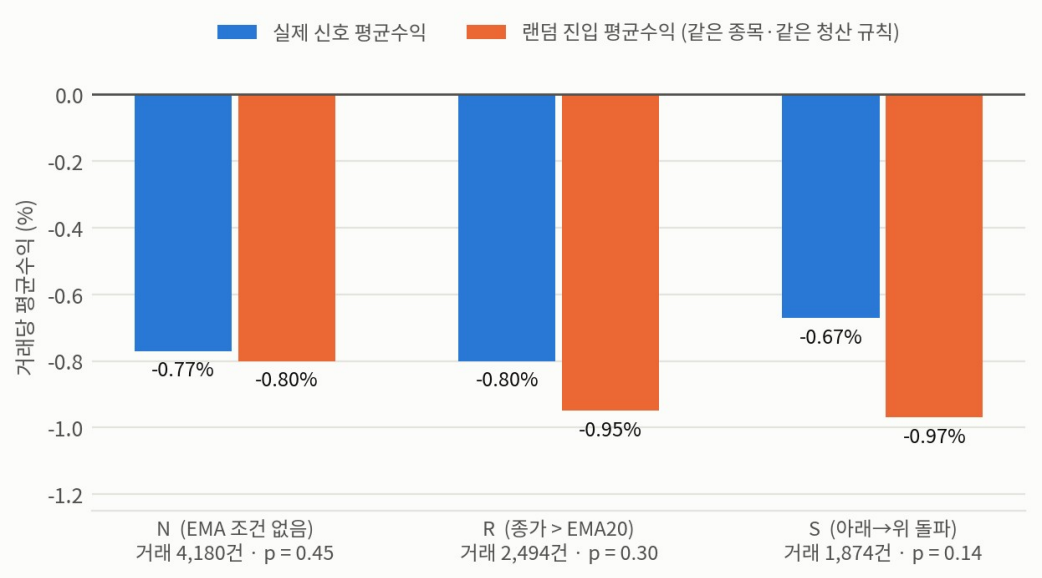


그림 3. 유동성 통과 신호의 평균수익(파랑)과 같은 종목·같은 청산 규칙의 랜덤 진입 평균(주황).

−0.8~−1.0%인 것은 비용·갭 손절·좁은 손절 구조 때문이며, 신호는 여기에 비해 0.03~0.3%p 낫지만 유의하지 않다. 승률은 31~32%로 손익비 2:1 의 손익분기(약 33%)에 못 미친다.

구간 분리(전체 신호 기준): 2024 년 전 / 후 평균수익이 N −0.04% / −0.31%, R −0.27% / −0.16%, S −0.16% / −0.01% 로 모든 구간이 0 이하였다.현재는 상장폐지된 종목만을 대상으로 하더라도, −0.75%~−0.95%였다.

③ 손절폭을 2 배로 넓히면?

손절이 너무 좁아서 잡음에 걸린다면 넓히면 나아질 수 있다는 가설이다. 새 손절가 = 진입가 − 2 × (진입가 − 기존 손절가), 평균 손절폭은 약 9%에서 약 18%가 됐다. **거의 모든 조합이 더 나빠졌다.** 예를 들어 N 필터 없음 손익비 2.0 은 −0.77%에서 −1.55%, S 는 −0.67%에서 −1.70%로 내려갔다. 한 번 질 때 손실이 두 배가 되는 효과를 승률 상승이 만회하지 못했다.

④ 코스피 20 일선 / 60 일선 필터

시장이 상승 국면일 때만 신호를 받으면 나아질 것이라는 가설이다. 신호 수는 절반 가까이 줄었지만 평균수익은 개선되지 않았다(예: N 손익비 2.0 에서 필터 없음 −0.77%, 20 일선 −0.92%, 60 일선 −0.81%). 무작위한 대조군도 같은 국면의 날에서만 뽑아 비교했다.

⑤ 손익비 0.5 ~ 2.0

"2 배 오르는 것보다 1 배 떨어지는 경우가 많으니 손익비를 낮추자"는 가설이다. 승률은 정확히 그렇게 움직였다 (31% → 61~64%). 그러나 평균수익은 같거나 더 나빴다. 같은 규칙의 무작위 진입도 승률이 똑같이 올랐기 때문에 **승률 상승은 우위의 증거가 아니라 손익비 선택의 결과**였다.

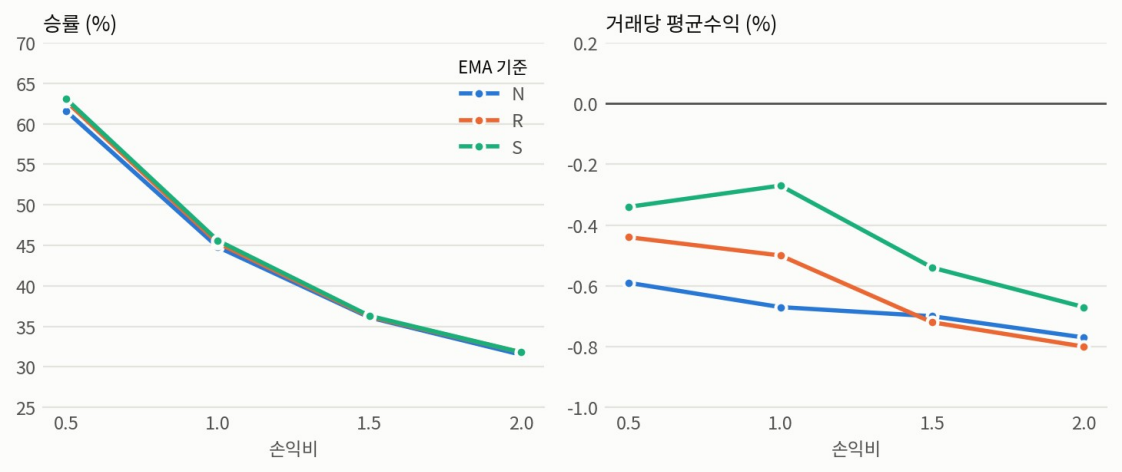


그림 4. 필터 없음, 손절폭 1 배 기준. 왼쪽: 손익비가 낮아질수록 승률은 61%대까지 오르고 세 기준이 거의 겹친다. 오른쪽: 평균수익은 모든 손익비에서 0 아래다.

	손절폭 1배 (기존)				손절폭 2배			
	손익비 (익절폭 ÷ 손절폭)							
	0.5	1.0	1.5	2.0	0.5	1.0	1.5	2.0
N · 필터 없음	-0.59	-0.67	-0.70	-0.77	-0.97	-1.25	-1.46	-1.55
N · 코스피>20일선	-0.68	-0.81	-0.97	-0.92	-1.23	-1.51	-1.59	-1.66
N · 코스피>60일선	-0.65	-0.74	-0.81	-0.81	-1.12	-1.42	-1.43	-1.50
R · 필터 없음	-0.44	-0.50	-0.72	-0.80	-0.94	-1.34	-1.48	-1.70
R · 코스피>20일선	-0.50	-0.64	-0.80	-0.73	-1.13	-1.27	-1.36	-1.50
R · 코스피>60일선	-0.50	-0.55	-0.61	-0.55	-0.99	-1.14	-1.10	-1.33
S · 필터 없음	-0.34	-0.27	-0.54	-0.67	-0.73	-1.25	-1.44	-1.70
S · 코스피>20일선	-0.43	-0.48	-0.60	-0.57	-1.04	-1.26	-1.25	-1.32
S · 코스피>60일선	-0.38	-0.35	-0.37	-0.37	-0.85	-1.07	-0.90	-1.07

그림 5. 72 개 조합의 거래당 평균수익(%). 칸 값은 모두 음수이며, 오른쪽(손절폭 2 배)으로 갈수록 붉은색이 짙어진다. 가장 좋은 칸도 −0.27%(S, 필터 없음, 손익비 1.0, PF 0.95)다.

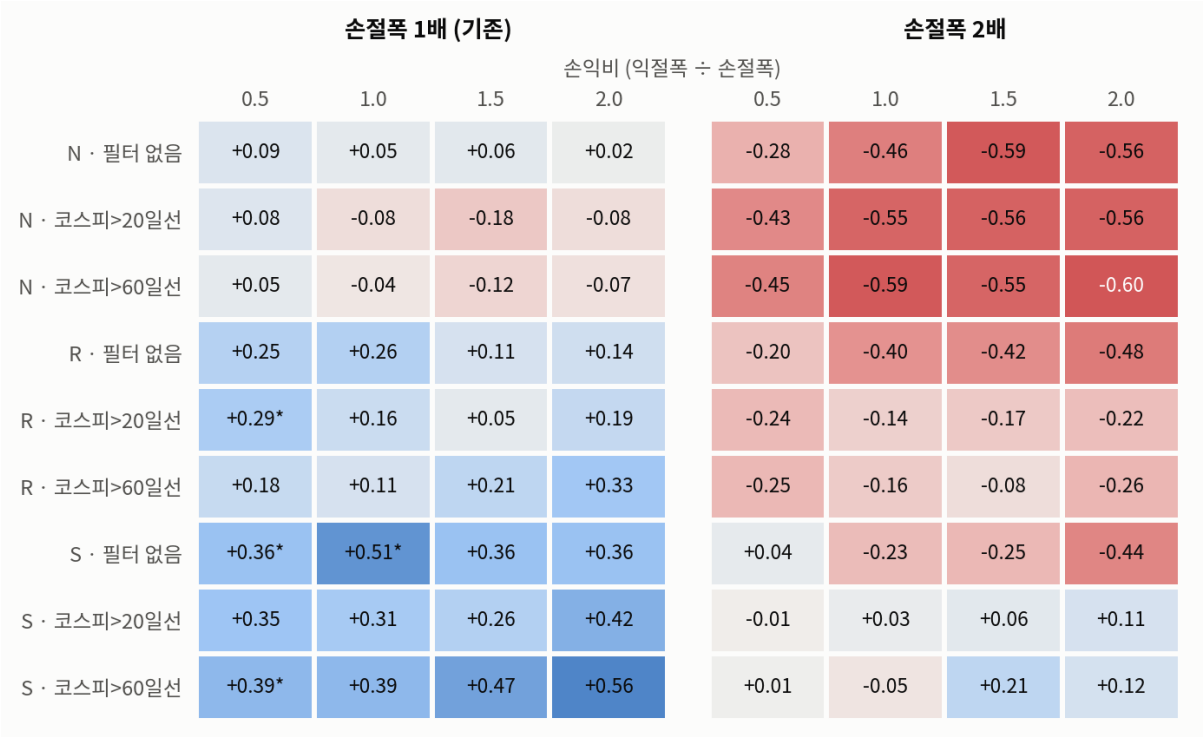


그림 6. 같은 조합에서 신호 평균수익 − 랜덤 평균수익(%p). 손절폭 1 배의 S 와 R 은 +0.1~+0.6%p 로 양수지만 절대수익은 음수이고, $p \le 0.05$(*)인 칸은 4 개뿐이다. 72 개를 비교하면 우연만으로도 약 3.6 개가 나오는 수준이다. 손절폭 2 배에서는 대부분 음수로 바뀐다.

⑥ 청산 규칙을 전부 빼고 보기: 고정 보유기간

손절·익절·히말라야 청산이 결과를 왜곡할 수 있으므로, 다음날 시가에 사서 5/10/20 일 뒤 종가에 파는 수익률을 같은 종목의 무작위 날과 비교했다. 같은 종목끼리 비교해서 종목 자체의 흐름을 제거했다.

27 개 조합 중 25 개에서 초과수익이 음수였다. 양수인 두 칸(+0.09%p, +0.07%p)은 t 값이 0.2 이하다. 기준 N 의 5 일·10 일 보유에서는 t 값이 약 −2.4 ~ −2.6 으로, 랜덤보다 **유의한 수준으로 나쁜수준이었다**. 해석 가설로, "하락추세 후 도지, 양봉" 모양은 반등의 시작보다 하락 도중의 일시적 쉼일 때가 많을 수 있다. 이 가설은 이 데이터만으로 검증되지는 않았다.

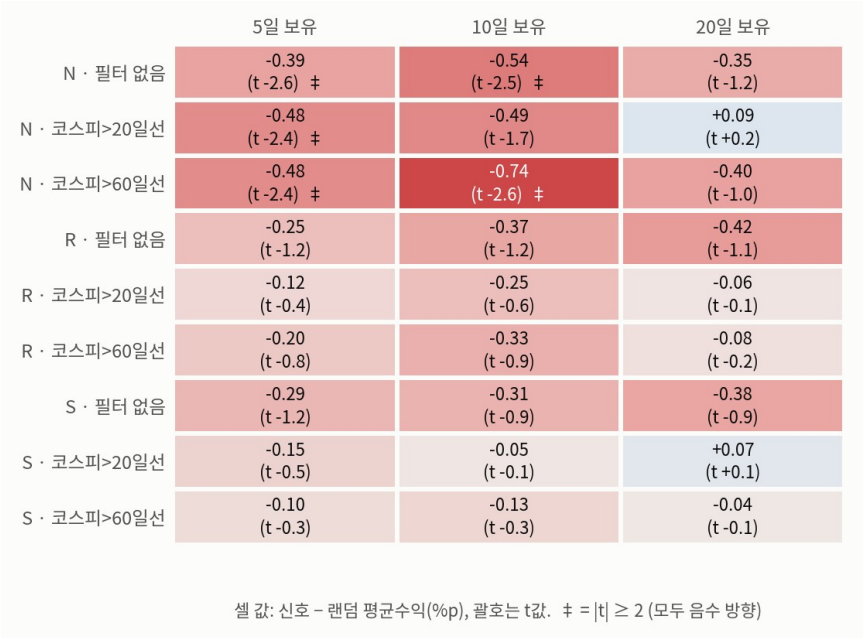


그림 7. 고정 보유기간 결과. 모든 신호 수는 1,037 건 이상이다. EMA20 조건(R, S)은 손해를 줄이는 정도이고 양수로 만들지 못했다.

---

4. 결론

- 비용을 반영한 평균수익이 72 개 조합 모두 마이너스다.
- 손절폭 · 손익비 · 시장 필터를 바꿔도 같은 종목의 랜덤 진입에 비해 믿을 만한 우위가 없다($p \le 0.05$는 72 개 중 4 개, 우연 기대치 3.6 개).
- 청산 규칙을 전부 빼도 결과가 같다. 문제는 청산이 아니라 **진입 신호 자체에 정보가 없다**는 점이다.
- 표본을 늘리자 가장 좋아 보이던 결과(S, 300 종목)가 사라졌다.

따라서 이 규칙을 그대로 매수 트리거로 쓰는 것은 근거가 없다. 스캐너는 후보를 추리는 보조 도구로만 쓰고, 최종 판단은 차트 맥락(지지·저항, 거래량), 섹터, 시장 상황으로 해야 한다.

---

5. 한계

- 일봉 기준만 백테스트했다(주봉 스캐너는 있으나 백테스트하지 않았다).
- 시장 필터는 코스피 지수 하나만 사용했다.
- 거래대금은 종가 × 거래량 근사이고, 가격은 pykrx 수정주가다.
- 상폐 종목은 분기별 상장 목록의 차이로 찾아서 일부 누락 가능성이 있다.
- 동시 보유 포트폴리오 시뮬레이션은 근사치라 해석에서 제외했다.
- 같은 신호가 여러 조합에 공유되므로 조합 간 결과는 서로 독립이 아니다.
- 차트 맥락을 읽는 재량 판단의 효과는 검증하지 못했다.

---

6. 시사하는 점

- 하이킨아시는 승률과 손익비는 서로 교환되는 경우가 많았다.
- 조합을 많이 시험할수록 우연한 성공이 늘어난다. 시험한 개수를 함께 보고한다.
- 검증 구간을 보고 파라미터를 다시 조정하면 검증의 의미가 사라진다.

- 이 틀(고정 보유기간 + 같은 종목 랜덤 비교)은 다른 신호에도 그대로 쓸 수 있어서, 다음 아이디어를 빠르게 걸러내는 도구가 된다.

---

7. 재현과 결과 파일

실행 파일	하는 일	만드는 결과
heikin_scanner_v2.py	일봉·주봉 신호 스캔, 보유종목 점검	signals_result_v2.xlsx
backtest_heikin_v2.py	N/R/S 기본 백테스트, 구간 분리, 랜덤 대조군	콘솔 출력, trades_v2.csv
backtest_sweep_v2.py	손절폭 × 손익비 × 코스피 필터 72 개 조합	sweep_summary_v2.csv
backtest_forward_v2.py	고정 보유기간 5/10/20 일, 같은 종목 랜덤 비교	forward_summary_v2.csv
screener_plus_v2.py	후보에 거래대금·손절폭·권장비중 등 보조정보	signals_enriched_v2.xlsx

GitHub 저장소의 results/ 폴더에 요약 CSV 와 콘솔 출력이 있다. 아래 부록에는 같은 숫자를 표로 옮겼다. 학습·연구용이며 투자 권유가 아니다.

부록 A. 손절폭 × 손익비 × 필터 72 개 조합

칸 값 = 거래당 평균수익(%), 아래 줄 = 승률(%) · 신호−랜덤 초과(%p). * = 랜덤 대조군 p ≤ 0.05. 유동성 통과 신호, 거래비용 0.3% 반영. 거래 수는 조합마다 1,046~4,250 건.

변형 · 필터	손절폭	손익비 0.5	손익비 1.0	손익비 1.5	손익비 2.0
N · 필터 없음	1 배	**−0.59%** 승률 61.6 · 초과 +0.09	**−0.67%** 승률 44.8 · 초과 +0.05	**−0.70%** 승률 36.1 · 초과 +0.06	**−0.77%** 승률 31.5 · 초과 +0.02
	2 배	**−0.97%** 승률 53.5 · 초과 −0.28	**−1.25%** 승률 38.3 · 초과 −0.46	**−1.46%** 승률 33.1 · 초과 −0.59	**−1.55%** 승률 31.3 · 초과 −0.56
N · 코스피>20 일선	1 배	**−0.68%** 승률 61.3 · 초과 +0.08	**−0.81%** 승률 44.5 · 초과 −0.08	**−0.97%** 승률 35.1 · 초과 −0.18	**−0.92%** 승률 30.5 · 초과 −0.08
	2 배	**−1.23%** 승률 52.8 · 초과 −0.43	**−1.51%** 승률 36.9 · 초과 −0.55	**−1.59%** 승률 31.8 · 초과 −0.56	**−1.66%** 승률 29.8 · 초과 −0.56
N · 코스피>60 일선	1 배	**−0.65%** 승률 61.5 · 초과 +0.05	**−0.74%** 승률 45.2 · 초과 −0.04	**−0.81%** 승률 35.9 · 초과 −0.12	**−0.81%** 승률 31.2 · 초과 −0.07
	2 배	**−1.12%** 승률 53.6 · 초과 −0.45	**−1.42%** 승률 37.3 · 초과 −0.59	**−1.43%** 승률 32.2 · 초과 −0.55	**−1.50%** 승률 30.2 · 초과 −0.60
R · 필터 없음	1 배	**−0.44%** 승률 62.8 · 초과 +0.25	**−0.50%** 승률 45.3 · 초과 +0.26	**−0.72%** 승률 36.2 · 초과 +0.11	**−0.80%** 승률 31.8 · 초과 +0.14
	2 배	**−0.94%** 승률 50.3 · 초과 −0.20	**−1.34%** 승률 35.8 · 초과 −0.40	**−1.48%** 승률 31.3 · 초과 −0.42	**−1.70%** 승률 29.9 · 초과 −0.48
R · 코스피>20 일선	1 배	**−0.50%** 승률 63.2 · 초과 +0.29*	**−0.64%** 승률 45.1 · 초과 +0.16	**−0.80%** 승률 36.0 · 초과 +0.05	**−0.73%** 승률 31.5 · 초과 +0.19
	2 배	**−1.13%** 승률 50.1 · 초과 −0.24	**−1.27%** 승률 35.6 · 초과 −0.14	**−1.36%** 승률 30.7 · 초과 −0.17	**−1.50%** 승률 28.9 · 초과 −0.22
R · 코스피>60 일선	1 배	**−0.50%** 승률 63.2 · 초과 +0.18	**−0.55%** 승률 46.2 · 초과 +0.11	**−0.61%** 승률 37.0 · 초과 +0.21	**−0.55%** 승률 32.8 · 초과 +0.33
	2 배	**−0.99%** 승률 51.4 · 초과 −0.25	**−1.14%** 승률 36.4 · 초과 −0.16	**−1.10%** 승률 31.9 · 초과 −0.08	**−1.33%** 승률 30.1 · 초과 −0.26
S · 필터 없음	1 배	**−0.34%** 승률 63.1 · 초과 +0.36*	**−0.27%** 승률 45.6 · 초과 +0.51*	**−0.54%** 승률 36.3 · 초과 +0.36	**−0.67%** 승률 31.8 · 초과 +0.36
	2 배	**−0.73%** 승률 51.0 · 초과 +0.04	**−1.25%** 승률 36.0 · 초과 −0.23	**−1.44%** 승률 31.6 · 초과 −0.25	**−1.70%** 승률 30.2 · 초과 −0.44
S · 코스피>20 일선	1 배	**−0.43%** 승률 63.5 · 초과 +0.35	**−0.48%** 승률 45.1 · 초과 +0.31	**−0.60%** 승률 36.5 · 초과 +0.26	**−0.57%** 승률 31.8 · 초과 +0.42
	2 배	**−1.04%** 승률 50.2 · 초과 −0.01	**−1.26%** 승률 35.8 · 초과 +0.03	**−1.25%** 승률 31.3 · 초과 +0.06	**−1.32%** 승률 29.7 · 초과 +0.11
S · 코스피>60 일선	1 배	**−0.38%** 승률 63.7 · 초과 +0.39*	**−0.35%** 승률 46.9 · 초과 +0.39	**−0.37%** 승률 37.7 · 초과 +0.47	**−0.37%** 승률 33.1 · 초과 +0.56
	2 배	**−0.85%** 승률 52.3 · 초과 +0.01	**−1.07%** 승률 36.9 · 초과 −0.05	**−0.90%** 승률 32.7 · 초과 +0.21	**−1.07%** 승률 31.0 · 초과 +0.12

부록 B. 고정 보유기간 27 개 조합

평균수익은 비용 0.3% 차감 후. 초과 = 신호 평균 − 같은 종목 랜덤 평균(%p). ‡ = |t| ≥ 2. 같은 종목에서 20 거래일 이내 재신호는 하나로 묶었다.

변형 · 필터	보유	신호 수	신호 평균	랜덤 평균	초과 %p	t 값
N · 없음	5 일	4,051	−1.12%	−0.72%	−0.39	−2.58 ‡
	10 일	4,037	−1.44%	−0.90%	−0.54	−2.54 ‡
	20 일	4,018	−1.69%	−1.35%	−0.35	−1.20
N · 코스피>20 일선	5 일	2,209	−1.29%	−0.81%	−0.48	−2.37 ‡
	10 일	2,195	−1.63%	−1.14%	−0.49	−1.68
	20 일	2,184	−1.73%	−1.82%	+0.09	+0.22
N · 코스피>60 일선	5 일	2,415	−1.17%	−0.69%	−0.48	−2.44 ‡
	10 일	2,400	−1.71%	−0.97%	−0.74	−2.60 ‡
	20 일	2,391	−1.90%	−1.50%	−0.40	−1.01
R · 없음	5 일	2,464	−1.02%	−0.77%	−0.25	−1.20
	10 일	2,452	−1.44%	−1.07%	−0.37	−1.25
	20 일	2,436	−2.00%	−1.58%	−0.42	−1.08
R · 코스피>20 일선	5 일	1,423	−0.98%	−0.87%	−0.12	−0.42
	10 일	1,410	−1.43%	−1.18%	−0.25	−0.63
	20 일	1,403	−1.95%	−1.89%	−0.06	−0.11
R · 코스피>60 일선	5 일	1,583	−0.91%	−0.72%	−0.20	−0.75
	10 일	1,570	−1.41%	−1.08%	−0.33	−0.87
	20 일	1,564	−1.79%	−1.70%	−0.08	−0.17
S · 없음	5 일	1,858	−1.09%	−0.80%	−0.29	−1.22
	10 일	1,851	−1.34%	−1.04%	−0.31	−0.90
	20 일	1,838	−1.96%	−1.58%	−0.38	−0.86
S · 코스피>20 일선	5 일	1,050	−0.99%	−0.83%	−0.15	−0.48
	10 일	1,043	−1.27%	−1.22%	−0.05	−0.11
	20 일	1,037	−1.86%	−1.92%	+0.07	+0.11
S · 코스피>60 일선	5 일	1,165	−0.87%	−0.77%	−0.10	−0.32
	10 일	1,158	−1.23%	−1.10%	−0.13	−0.29
	20 일	1,152	−1.84%	−1.80%	−0.04	−0.06

부록 C. 소스 코드

이 보고서의 모든 결과를 만든 코드 전체다. 같은 폴더에 두고 실행하며, 각 파일의 BASE_DIR 은 그 파일이 있는 폴더로 자동 설정된다. 실행 순서는 7 절의 표를 따른다. 가격과 상장 목록은 pykrx 로 수집하고 KRX 로그인 정보는 환경변수(KRX_ID, KRX_PW)로만 사용하며 코드에는 포함하지 않는다. 입력 파일(stock_universe.csv, sector1.csv)은 대회 제공 자료이므로 저장소에 포함하지 않았다.

C-1. heikin_scanner_v2.py (신호 스캐너: 규칙 구현, 일봉·주봉 스캔, 보유종목 점검)

```
# -*- coding: utf-8 -*-
"""
하이킨아시 스윙 스캐너 v2 + 보유종목 관리  (v1 heikin_scanner.py 는 그대로 두고 따로 쓰는 버전)
================================================
필수 4 조건 (모두 만족해야 신호, 가점 없음)
  (1) 도지 연속 후 평평한 바닥 + 긴 윗꼬리 양봉 (일반 하이킨아시)
  (2) 신호봉 거래량 > 직전 도지 거래량
  (3) 도지 직전 하락추세 (음봉 비율)
  (4) 신호봉에서 EMA20 돌파 -> 기준 2 가지를 모두 표시 (설정 SCAN_EMA_MODE 로 통과 기준 선택)
        확실: 전날 종가는 EMA20 아래, 신호봉 종가는 EMA20 위 (아래->위 돌파)
        완화: 신호봉 종가가 EMA20 위
매도: 손절가 / 익절가 도달, 또는 스무스 하이킨아시(히말라야) 양봉->음봉 전환 즉시 매도
      (익절가에 못 가도 전환되면 매도)
================================================
사용법 (Spyder 기준):
  1. 이 .py 파일들을 한 폴더에 둔다 (BASE_DIR 은 자동으로 그 폴더가 된다).
  2. 그 폴더에 stock_universe.csv, sector1.csv (보유종목 점검을 쓰면 holdings.xlsx 도) 를 둔다.
  3. Run File(F5)로 돌린다. 작업 폴더가 어디든 상관없이 BASE_DIR 안의 파일만 읽고 쓴다.
  4. 결과는 BASE_DIR 안의 signals_result_v2.xlsx 로 저장되고, 콘솔에도 요약이 출력된다.
처음엔 TEST_MODE=True 로 소수 종목만 테스트해보는 걸 추천.
"""

import os
import pandas as pd
import numpy as np
import time
from datetime import datetime, timedelta

from pykrx import stock
import pandas_ta as ta

# ============================ [설정] ============================
# 이 프로젝트 폴더 = 이 파일이 있는 폴더 (파일 읽기/쓰기는 전부 이 폴더 안에서 함). 폴더를 옮겨도 자동으로 따라감
BASE_DIR = os.path.dirname(os.path.abspath(__file__))   # 이 파일이 있는 폴더 (어디서 실행해도 동일)


def P(filename):
    return os.path.join(BASE_DIR, filename)


UNIVERSE_PATH = P('stock_universe.csv')
SECTOR_PATH = P('sector1.csv')
OUTPUT_PATH = P('signals_result_v2.xlsx')

LOOKBACK_DAYS = 1500       # 데이터 조회 기간 (주봉 지표 워밍업까지 고려해 약 5 년)
HA_LEN = 10                 # 1 차 하이킨아시 평활화 길이 (파인스크립트 len)
HA_LEN2 = 10                # 2 차 평활화 길이 (파인스크립트 len2)

DOJI_BODY_PCT = 0.10        # 몸통 <= 레인지의 10% 이면 도지 (일반 하이킨아시 기준)
FLAT_WICK_PCT = 0.10        # 아래꼬리 <= 레인지의 10% 이면 "평평" (일반 하이킨아시 기준)
LONG_UPPER_WICK_PCT = 0.30  # 윗꼬리 >= 레인지의 30% 이면 "길게 위로 찌름" (일반 하이킨아시 기준)

EMA_TREND_LEN = 20          # 돌파 기준 EMA
# 신호 통과 기준 (필수 4 번째 조건): 'strict' = 신호봉에서 아래->위 돌파만 / 'relaxed' = 종가가 EMA20 위면 통과 / 'none' = EMA 조건 없음
# 어느 쪽이든 결과 엑셀에는 '확실 돌파' 여부가 O/- 로 같이 표시되므로 relaxed 로 두고 확실한 것만 골라 봐도 됨
SCAN_EMA_MODE = 'relaxed'

USE_DAILY = True            # 일봉 스캔 켜기/끄기
USE_WEEKLY = True           # 주봉 스캔 켜기/끄기 (이번 주 진행 중인 봉도 포함)
TREND_WINDOW = 12           # [일봉] 첫 도지 직전 몇 개 봉을 보고 '하락추세였는지' 판단할지
TREND_WINDOW_W = 8          # [주봉] 같은 기준 (8 주)
TREND_BEARISH_RATIO = 0.65  # 그 구간에서 음봉 비율이 이 값 이상이면 하락추세로 인정
STOP_BUFFER = 0.005         # 손절가 = 도지들 최저 저가 x (1 - 0.5%)
```

```
RR_RATIO = 2.0              # 손익비 (익절 = 진입 + 2 x (진입-손절))
HOLDINGS_PATH = P('holdings.xlsx')  # 보유종목 엑셀 (직접 입력)
HOLDINGS_ONLY = False       # True 면 신규신호 스캔은 건너뛰고 보유종목만 점검 (결과: holdings_result_v2.xlsx)

RECHECK_WINDOW_W = 2        # [주봉] 최근 몇 주 내 신호를 눌림목 재탐색 대상으로 볼지
RECHECK_WINDOW = 5          # 최근 며칠 내 신호를 "눌림목 재탐색" 대상으로 볼지
SIGNAL_LOOKBACK_FOR_SCAN = 1  # 오늘(가장 최근 봉) 신규 신호만 볼지(1), 더 늘리면 최근 n 봉 전부 스캔

# --- 매수 불가 종목 필터 (대회 규정) ---
MIN_MARKET_CAP = 100_000_000_000       # 시가총액 1,000 억원 미만 제외
MIN_AVG_TRADING_VALUE = 3_000_000_000  # 최근 5 거래일 평균 거래대금 30 억원 이하 제외
TRADING_VALUE_AVG_DAYS = 5             # 거래대금 평균 낼 거래일 수

# True  -> 대회 조건(시총/거래대금) 미달 종목은 아예 스캔 대상에서 빼버림 (대회용, 결과에 대회 조건 미달 종목 없음)
# False -> 전부 다 스캔하고, 결과 엑셀에 '대회매수가능' 컬럼(적합/부적합)만 추가로 붙여줌 (개인투자 겸용)
EXCLUDE_INELIGIBLE = False

TEST_MODE = True            # True 면 앞의 TEST_N 개 종목만 테스트
TEST_N = 30

SLEEP_SEC = 0.15            # pykrx 과호출 방지용 딜레이(초)
# ================================================================


def load_universe():
    df = pd.read_csv(UNIVERSE_PATH, dtype=str)
    df['pykrx_code'] = df['종목코드'].str.replace('^A', '', regex=True)
    return df


def load_sector_limits():
    df = pd.read_csv(SECTOR_PATH)
    return df[['섹터코드', '섹터명', '시장 비중', '한도 비중']]


def get_eligible_codes(end):
    """
    '매수 불가 종목' 중 시가총액 / 거래대금 조건을 코드로 거를 수 있는 종목코드 집합을 반환.
    - 시가총액 1,000 억원 미만 제외
    - 최근 5 거래일 평균 거래대금 30 억원 이하 제외
    종목 단위로 하나씩 조회하지 않고, '날짜 하나당 전체 시장 스냅샷'을 받아오는 방식이라
    종목이 2,600 개가 넘어도 API 호출은 며칠치(기본 5 번)만 하면 됨 -> 훨씬 빠름.
    실패하면 (필터를 건너뛴다는 의미로) None 을 반환.
    """
    base_date = None
    cap_today = None
    d = datetime.strptime(end, '%Y%m%d')
    for _ in range(10):
        ds = d.strftime('%Y%m%d')
        try:
            tmp = stock.get_market_cap_by_ticker(ds, market="ALL")
            if tmp is not None and len(tmp) > 0:
                cap_today = tmp
                base_date = d
                break
        except Exception as e:
            print(f"  [시총조회 실패] {ds}: {e}")
        d -= timedelta(days=1)

    if cap_today is None:
        print("[경고] 시가총액 데이터를 가져오지 못해 시총/거래대금 필터를 건너뜁니다.")
        return None

    value_frames = []
    d = base_date
    tries = 0
    while len(value_frames) < TRADING_VALUE_AVG_DAYS and tries < 15:
        ds = d.strftime('%Y%m%d')
        try:
            tmp = stock.get_market_cap_by_ticker(ds, market="ALL")
            if tmp is not None and len(tmp) > 0:
                value_frames.append(tmp['거래대금'])
        except Exception as e:
            print(f"  [거래대금조회 실패] {ds}: {e}")
        d -= timedelta(days=1)
        tries += 1

    eligible = set(cap_today.index[cap_today['시가총액'] >= MIN_MARKET_CAP])
```

```
    if value_frames:
        avg_value = pd.concat(value_frames, axis=1).mean(axis=1)
        eligible &= set(avg_value.index[avg_value >= MIN_AVG_TRADING_VALUE])
    else:
        print("[경고] 거래대금 데이터를 가져오지 못해 거래대금 필터는 건너뜁니다.")

    return eligible


def fetch_ohlcv(code, start, end):
    """pykrx로 일봉 OHLCV 가져오기. 실패하면 None 반환."""
    try:
        df = stock.get_market_ohlcv(start, end, code)
        if df is None or len(df) < 60:
            return None
        df = df.rename(columns={
            '시가': 'open', '고가': 'high', '저가': 'low',
            '종가': 'close', '거래량': 'volume'
        })
        return df[['open', 'high', 'low', 'close', 'volume']]
    except Exception:
        return None


def heikin_ashi_raw(df):
    """일반(평활화 없는) 하이킨아시. 도지/평평한바닥 패턴 탐지에 사용."""
    o = df['open']
    h = df['high']
    l = df['low']
    c = df['close']

    haclose = (o + h + l + c) / 4
    haopen = pd.Series(index=df.index, dtype=float)
    haopen.iloc[0] = (o.iloc[0] + c.iloc[0]) / 2
    for i in range(1, len(df)):
        haopen.iloc[i] = (haopen.iloc[i - 1] + haclose.iloc[i - 1]) / 2

    hahigh = pd.concat([h, haopen, haclose], axis=1).max(axis=1)
    halow = pd.concat([l, haopen, haclose], axis=1).min(axis=1)

    out = pd.DataFrame({'o1': haopen, 'h1': hahigh, 'l1': halow, 'c1': haclose}, index=df.index)
    return out


def smoothed_heikin_ashi(df, len1=HA_LEN, len2=HA_LEN2):
    """파인스크립트 'Smoothed Heiken Ashi Candles' 그대로 복제."""
    o = ta.ema(df['open'], length=len1)
    c = ta.ema(df['close'], length=len1)
    h = ta.ema(df['high'], length=len1)
    l = ta.ema(df['low'], length=len1)

    haclose = (o + h + l + c) / 4
    haopen = pd.Series(index=df.index, dtype=float)
    haopen.iloc[0] = (o.iloc[0] + c.iloc[0]) / 2 if not np.isnan(o.iloc[0]) else np.nan
    for i in range(1, len(df)):
        if np.isnan(haopen.iloc[i - 1]):
            haopen.iloc[i] = (o.iloc[i] + c.iloc[i]) / 2
        else:
            haopen.iloc[i] = (haopen.iloc[i - 1] + haclose.iloc[i - 1]) / 2

    hahigh = pd.concat([h, haopen, haclose], axis=1).max(axis=1)
    halow = pd.concat([l, haopen, haclose], axis=1).min(axis=1)

    o2 = ta.ema(haopen, length=len2)
    c2 = ta.ema(haclose, length=len2)
    h2 = ta.ema(hahigh, length=len2)
    l2 = ta.ema(halow, length=len2)

    out = pd.DataFrame({'o2': o2, 'h2': h2, 'l2': l2, 'c2': c2}, index=df.index)
    return out


def add_indicators(df):
    df = df.copy()
    df['ema20'] = ta.ema(df['close'], length=EMA_TREND_LEN)
    return df
```

```
def tick_size(price):
    """한국거래소 주식 호가가격단위 (2023.1.25~ 코스피/코스닥 공통, ETF 제외)."""
    if price < 2000:
        return 1
    if price < 5000:
        return 5
    if price < 20000:
        return 10
    if price < 50000:
        return 50
    if price < 200000:
        return 100
    if price < 500000:
        return 500
    return 1000


def round_to_tick(price):
    """가격을 호가단위에 맞춰 반올림 (주문창에 그대로 입력할 수 있게)."""
    t = tick_size(price)
    return int(round(price / t) * t)


def find_signals(df, trend_window, ema_mode=None):
    """
    필수 4 조건을 모두 만족하는 봉을 전부 찾아서 리스트로 반환.
      ① 직전 봉(들)이 도지 -> 현재 봉이 양봉 + 아래꼬리 평평 + 윗꼬리 김 (일반 하이킨아시)
      ② 현재 봉(신호 봉) 거래량 > 바로 직전 도지 봉 거래량
      ③ 연속 도지가 시작되기 직전 trend_window 봉 중 음봉 비율 >= TREND_BEARISH_RATIO (하락 추세)
      ④ EMA20 돌파 (ema_mode: 'strict' 아래->위 돌파 / 'relaxed' 종가가 EMA20 위 / 'none' 조건 없음)
    진입가 = 신호일 실제 종가
    손절가 = 연속 도지 전체의 실제 최저가 x (1 - STOP_BUFFER)
    익절가 = 진입가 + RR_RATIO x (진입가 - 손절가)
    """
    if ema_mode is None:
        ema_mode = SCAN_EMA_MODE
    n = len(df)
    rng = (df['h1'] - df['l1']).to_numpy()
    body = (df['c1'] - df['o1']).abs().to_numpy()
    lower_wick = (df[['o1', 'c1']].min(axis=1) - df['l1']).to_numpy()
    upper_wick = (df['h1'] - df[['o1', 'c1']].max(axis=1)).to_numpy()

    valid = rng > 0
    is_doji = valid & (body <= DOJI_BODY_PCT * rng)
    is_flat = valid & (lower_wick <= FLAT_WICK_PCT * rng)
    is_long_up = valid & (upper_wick >= LONG_UPPER_WICK_PCT * rng)
    is_bull = (df['c1'] > df['o1']).to_numpy()
    is_bear = (df['c1'] < df['o1']).to_numpy()

    close = df['close'].to_numpy()
    low = df['low'].to_numpy()
    volume = df['volume'].to_numpy()
    ema20 = df['ema20'].to_numpy()

    # 각 봉까지 이어진 연속 도지 개수
    doji_run = np.zeros(n, dtype=int)
    for i in range(n):
        if is_doji[i]:
            doji_run[i] = doji_run[i - 1] + 1 if i > 0 else 1

    signals = []
    for i in range(1, n):
        if not (is_bull[i] and is_flat[i] and is_long_up[i]):
            continue
        run = doji_run[i - 1]
        if run == 0:                        # 직전 봉이 도지가 아님
            continue
        s = i - run                         # 연속 도지의 첫 봉 위치
        if not volume[i] > volume[i - 1]:  # 거래량: 신호 봉 > 바로 직전 도지
            continue
        w0 = s - trend_window
        if w0 < 0:
            continue
        bear_ratio = is_bear[w0:s].mean()  # 도지 구간 직전 trend_window 봉의 음봉 비율
        if bear_ratio < TREND_BEARISH_RATIO:
            continue
```

```
        ema_above = bool(close[i] > ema20[i])                          # 완화: 종가가 EMA20 위
        ema_cross = bool(close[i - 1] < ema20[i - 1] and close[i] > ema20[i])   # 확실: 아래->위 돌파
        if ema_mode == 'strict' and not ema_cross:
            continue
        if ema_mode == 'relaxed' and not ema_above:
            continue

        entry = int(close[i])
        stop = round_to_tick(low[s:i].min() * (1 - STOP_BUFFER))
        if stop >= entry:
            continue
        target = round_to_tick(entry + RR_RATIO * (entry - stop))

        signals.append({
            'idx': i, 'entry': entry, 'stop': stop, 'target': target,
            'doji_n': int(run), 'bear_ratio': float(bear_ratio),
            'ema_cross': ema_cross, 'ema_above': ema_above,
        })
    return signals


def to_weekly(raw):
    """일봉 -> 주봉 (금요일 기준 주 단위). 시가=첫날, 고가=최대, 저가=최소, 종가=마지막날, 거래량=합계."""
    w = raw[['open', 'high', 'low', 'close', 'volume']].resample('W-FRI').agg(
        {'open': 'first', 'high': 'max', 'low': 'min', 'close': 'last', 'volume': 'sum'})
    return w.dropna(subset=['open', 'close'])


def analyze_timeframe(raw, tf, trend_window, recheck_window, code, name, sector_code):
    """한 종목의 한 시간프레임(일봉/주봉)을 분석해서 결과 dict 리스트(또는 None) 반환."""
    raw = add_indicators(raw)
    ha1 = heikin_ashi_raw(raw)                              # 패턴/추세 판단용 (일반 하이킨아시)
    ha2 = smoothed_heikin_ashi(raw, HA_LEN, HA_LEN2)     # 매도 타이밍용 (스무스 하이킨아시)
    df = raw.join(ha1).join(ha2)
    df = df.dropna().copy()
    if len(df) < trend_window + 20:
        return None

    signals = find_signals(df, trend_window)
    if not signals:
        return None

    n = len(df)
    cur_close = int(df['close'].iloc[-1])
    # 스무스 하이킨아시: 직전 봉 양봉 -> 현재 봉 음봉 (오늘 막 전환됐으면 매도)
    ha2_color = '양봉(상승)' if df['c2'].iloc[-1] > df['o2'].iloc[-1] else '음봉(하락)'
    sell_now = bool((df['c2'].iloc[-2] > df['o2'].iloc[-2]) and (df['o2'].iloc[-1] > df['c2'].iloc[-1]))

    results = []
    for sg in signals:
        i = sg['idx']
        kind = None
        if i >= n - SIGNAL_LOOKBACK_FOR_SCAN:
            kind = '신규신호'
        elif (n - 1 - recheck_window) <= i <= n - 2:
            # 눌림목 재탐색: 최근 신호 후 손절가를 안 깼고, 현재가가 진입가 아래로 눌려있는 경우
            hit_stop = bool((df['low'].iloc[i:] <= sg['stop']).any())
            pulled_back = sg['stop'] < cur_close < sg['entry']
            if (not hit_stop) and pulled_back:
                kind = '눌림목재탐색'
        if kind is None:
            continue

        results.append({
            '종목코드': code, '종목명': name, '섹터코드': sector_code, '봉': tf,
            '신호일': df.index[i].date(), '구분': kind,
            'EMA20 확실돌파': 'O' if sg['ema_cross'] else '-',
            'EMA20 종가위': 'O' if sg['ema_above'] else '-',
            '도지개수': sg['doji_n'],
            '하락봉비율(%)': round(sg['bear_ratio'] * 100),
            '진입가': sg['entry'], '손절가': sg['stop'], '익절가': sg['target'],
            '현재가': cur_close,
            '스무스 HA 현재색': ha2_color,
            '스무스 HA 매도전환': sell_now,
        })

    return results if results else None
```

```
def analyze_one_stock(code, name, sector_code, start, end):
    raw = fetch_ohlcv(code, start, end)          # 일봉은 한 번만 내려받고 주봉은 여기서 변환
    if raw is None:
        return None
    out = []
    if USE_DAILY:
        r = analyze_timeframe(raw, '일봉', TREND_WINDOW, RECHECK_WINDOW, code, name, sector_code)
        if r:
            out.extend(r)
    if USE_WEEKLY:
        r = analyze_timeframe(to_weekly(raw), '주봉', TREND_WINDOW_W, RECHECK_WINDOW_W, code, name, sector_code)
        if r:
            out.extend(r)
    return out if out else None


# ====================== 보유 종목 점검 ======================
def load_holdings():
    """holdings.xlsx 를 읽어서 DataFrame 반환. 파일이 없거나 비어있으면 None."""
    if not os.path.exists(HOLDINGS_PATH):
        print(f"[보유종목] {HOLDINGS_PATH} 파일이 없어 보유종목 점검은 건너뜁니다.")
        return None

    h = pd.read_excel(HOLDINGS_PATH, dtype=str)
    h.columns = [str(c).strip() for c in h.columns]
    if '종목코드' not in h.columns:
        print(f"[보유종목] {HOLDINGS_PATH} 에 '종목코드' 열이 없어 건너뜁니다.")
        return None

    h = h.dropna(subset=['종목코드']).copy()
    if len(h) == 0:
        print("[보유종목] 보유종목 파일이 비어있어 점검은 건너뜁니다.")
        return None

    h['종목코드'] = (h['종목코드'].astype(str).str.strip()
                     .str.replace(r'\.0$', '', regex=True).str.upper().str.zfill(6))
    for col in ['종목명', '매수일', '매수가', '손절가', '익절가']:
        if col not in h.columns:
            h[col] = np.nan
    h['매수일'] = pd.to_datetime(h['매수일'], errors='coerce')
    for col in ['매수가', '손절가', '익절가']:
        h[col] = pd.to_numeric(h[col], errors='coerce')
    return h.reset_index(drop=True)


def check_holding(row, start, end):
    """보유 종목 1개 점검: 손절/익절 도달 또는 스무스 하이킨아시(히말라야) 양->음 전환이면 매도."""
    base = {
        '종목코드': row['종목코드'], '종목명': row['종목명'],
        '매수일': row['매수일'].date() if pd.notna(row['매수일']) else '',
        '매수가': row['매수가'], '손절가': row['손절가'], '익절가': row['익절가'],
    }
    raw = fetch_ohlcv(row['종목코드'], start, end)
    if raw is None:
        base.update({'현재가': '', '수익률(%)': '', '스무스 HA': '', '최근전환일': '', '상태': '데이터 조회 실패'})
        return base

    ha2 = smoothed_heikin_ashi(raw, HA_LEN, HA_LEN2)
    df = raw.join(ha2).dropna().copy()

    bull = df['c2'] > df['o2']
    flip = bull.shift(1, fill_value=False) & (df['o2'] > df['c2'])   # 양 -> 음 전환 봉

    # 매수일이 있으면 그날부터, 비어 있으면 오늘 봉 하나만 본다 (예전 데이터로 잘못 걸리는 것 방지)
    since = df[df.index >= row['매수일']] if pd.notna(row['매수일']) else df.iloc[-1:]

    cur = int(df['close'].iloc[-1])
    buy = row['매수가']
    ret = round((cur / buy - 1) * 100, 2) if pd.notna(buy) and buy > 0 else ''

    stop, target = row['손절가'], row['익절가']
    stop_date = None
    target_date = None
    if pd.notna(stop):
        hit = since.index[(since['low'] <= stop).to_numpy()]
        stop_date = hit[0] if len(hit) else None
    if pd.notna(target):
```

```
        hit = since.index[(since['high'] >= target).to_numpy()]
        target_date = hit[0] if len(hit) else None

    flip_dates = since.index[flip.loc[since.index].to_numpy()]
    last_flip = flip_dates[-1] if len(flip_dates) else None
    flip_today = bool(flip.iloc[-1])
    now_bull = bool(bull.iloc[-1])

    if stop_date is not None and (target_date is None or stop_date <= target_date):
        status = f"손절가 도달 ({stop_date.date()}) -> 매도"
    elif target_date is not None:
        status = f"익절가 도달 ({target_date.date()}) -> 매도"
    elif flip_today:
        status = "매도: 스무스 HA 양봉->음봉 전환 (오늘)"
    elif last_flip is not None:
        status = f"매도: 스무스 HA 음봉 전환 이미 발생 (전환일 {last_flip.date()})"
    else:
        status = "보유 유지"

    base.update({
        '현재가': cur, '수익률(%)': ret,
        '스무스 HA': '양봉(상승)' if now_bull else '음봉(하락)',
        '최근전환일': last_flip.date() if last_flip is not None else '',
        '상태': status,
    })
    return base


def main():
    if not os.path.isdir(BASE_DIR):
        print(f"[오류] BASE_DIR 폴더를 찾을 수 없어요: {BASE_DIR}\n  코드 위쪽 [설정]의 BASE_DIR 경로를 확인하세요.")
        return
    universe = load_universe()
    sectors = load_sector_limits()

    end = datetime.today().strftime('%Y%m%d')
    start = (datetime.today() - timedelta(days=LOOKBACK_DAYS)).strftime('%Y%m%d')

    if HOLDINGS_ONLY:
        print("[보유종목만 점검 모드] 신규신호 스캔은 건너뜁니다.")
        holdings = load_holdings()
        if holdings is None:
            return
        rows = []
        for _, hr in holdings.iterrows():
            try:
                rows.append(check_holding(hr, start, end))
            except Exception as e:
                print(f"  [보유종목 에러] {hr['종목명']}({hr['종목코드']}): {e}")
            time.sleep(SLEEP_SEC)
        if not rows:
            print("점검 결과가 없습니다.")
            return
        holding_df = pd.DataFrame(rows)
        try:
            holding_df.to_excel(P('holdings_result_v2.xlsx'), sheet_name='보유종목', index=False)
        except PermissionError:
            print("\n[저장 실패] holdings_result_v2.xlsx 가 엑셀에서 열려있어요. 닫고 다시 실행해주세요.")
            return
        print(f"\n완료! 보유종목 {len(holding_df)}건 점검 -> {P('holdings_result_v2.xlsx')} 저장됨\n")
        print(holding_df[['종목명', '현재가', '수익률(%)', '스무스 HA', '상태']].to_string(index=False))
        return

    eligible = get_eligible_codes(end)

    if EXCLUDE_INELIGIBLE:
        if eligible is not None:
            before_n = len(universe)
            universe = universe[universe['pykrx_code'].isin(eligible)].reset_index(drop=True)
            print(f"[매수가능 필터] {before_n}개 -> {len(universe)}개 "
                  f"(시총 {MIN_MARKET_CAP/1e8:.0f}억원 이상 & "
                  f"{TRADING_VALUE_AVG_DAYS}일평균거래대금 {MIN_AVG_TRADING_VALUE/1e8:.0f}억원 이상)")
    else:
        print("[매수가능 필터] EXCLUDE_INELIGIBLE=False -> 전종목 스캔, "
              "결과에 '대회매수가능' 컬럼으로만 표시합니다.")

    if TEST_MODE:
        universe = universe.head(TEST_N)
```

```
        print(f"[테스트 모드] {TEST_N}개 종목만 스캔합니다.")

    # ---------- 1) 신규 신호 스캔 ----------
    all_results = []
    n = len(universe)
    for i, r in enumerate(universe.itertuples(), 1):
        code, name, sector_code = r.pykrx_code, r.종목명, r.섹터코드
        try:
            res = analyze_one_stock(code, name, sector_code, start, end)
            if res:
                all_results.extend(res)
        except Exception as e:
            print(f"  [에러] {name}({code}): {e}")
        if i % 50 == 0 or i == n:
            print(f"진행 {i}/{n} ...")
        time.sleep(SLEEP_SEC)

    base_cols = ['종목코드', '종목명', '섹터코드', '봉', '신호일', '구분', 'EMA20 확실돌파', 'EMA20 종가위',
                 '도지개수', '하락봉비율(%)',
                 '진입가', '손절가', '익절가', '현재가', '스무스 HA 현재색', '스무스 HA 매도전환']
    if all_results:
        result_df = pd.DataFrame(all_results)
        result_df = result_df.merge(sectors, on='섹터코드', how='left')
        if eligible is not None:
            result_df['대회매수가능'] = np.where(result_df['종목코드'].isin(eligible), '적합', '부적합')
        else:
            result_df['대회매수가능'] = '확인불가'
        result_df['_k'] = (result_df['구분'] != '신규신호')
        result_df['_e'] = (result_df['EMA20 확실돌파'] != 'O')     # 확실 돌파가 위로
        result_df = result_df.sort_values(['_k', '_e', '신호일'], ascending=[True, True, False]).drop(columns=['_k', '_e'])
    else:
        print("조건에 맞는 신규 신호가 없습니다.")
        result_df = pd.DataFrame(columns=base_cols + ['섹터명', '시장 비중', '한도 비중', '대회매수가능'])

    # ---------- 2) 보유 종목 점검 ----------
    holding_df = None
    holdings = load_holdings()
    if holdings is not None:
        rows = []
        for _, hr in holdings.iterrows():
            try:
                rows.append(check_holding(hr, start, end))
            except Exception as e:
                print(f"  [보유종목 에러] {hr['종목명']}({hr['종목코드']}): {e}")
            time.sleep(SLEEP_SEC)
        if rows:
            holding_df = pd.DataFrame(rows)

    # ---------- 3) 저장 ----------
    try:
        with pd.ExcelWriter(OUTPUT_PATH, engine='openpyxl') as writer:
            for tf in ['일봉', '주봉']:
                part = result_df[result_df['봉'] == tf] if '봉' in result_df.columns else result_df.iloc[0:0]
                part.drop(columns='봉', errors='ignore').to_excel(writer, sheet_name=f'신규신호({tf})', index=False)
            if holding_df is not None:
                holding_df.to_excel(writer, sheet_name='보유종목', index=False)
    except PermissionError:
        print(f"\n[저장 실패] {OUTPUT_PATH} 가 엑셀에서 열려있어요. 닫고 다시 실행해주세요.")
        return

    n_d = int((result_df['봉'] == '일봉').sum()) if len(result_df) else 0
    n_w = int((result_df['봉'] == '주봉').sum()) if len(result_df) else 0
    print(f"\n완료! 신규신호 일봉 {n_d}건 / 주봉 {n_w}건"
          + (f", 보유종목 {len(holding_df)}건 점검" if holding_df is not None else "")
          + f" -> {OUTPUT_PATH} 저장됨")
    if len(result_df):
        print(result_df.head(20))
    if holding_df is not None:
        print("\n[보유종목 상태]")
        print(holding_df[['종목명', '현재가', '수익률(%)', '스무스 HA', '상태']].to_string(index=False))


if __name__ == '__main__':
    main()
```

C-2. backtest_heikin_v2.py (기본 백테스트: N / R / S 기준, 구간 분리, 랜덤 대조군)

```
# -*- coding: utf-8 -*-
"""
하이킨아시 스캐너 v2 백테스트 (heikin_scanner_v2.py 와 같은 폴더(BASE_DIR)에 두고 실행)
================================================================
heikin_scanner_v2.py 의 신호 탐지 함수(find_signals)와 설정값을 그대로 가져다 쓴다.

[규칙]
  진입: 필수 4 조건 (도지 후 평평한바닥+긴윗꼬리 양봉 / 거래량 / 직전 하락추세 / EMA20 돌파) -> 다음날 시가 진입 가정
  청산: 손절가, 익절가, 스무스 하이킨아시(히말라야) 양봉->음봉 전환 (전환 다음날 시가 매도), 최대보유일 만료 중 먼저 오는 것

[3 가지 EMA 기준을 동시에 비교]
  N) EMA 조건 없음 (3 조건만)  <- EMA20 돌파가 실제로 도움이 되는지 보는 기준선
  R) 완화: 신호봉 종가가 EMA20 위
  S) 확실: 전날 종가는 EMA20 아래, 신호봉 종가는 EMA20 위 (아래->위 돌파)

[포함된 것]
  - 상장폐지/합병으로 사라진 종목도 함께 백테스트 (생존편향 완화, pykrx 과거 상장목록 활용)
  - 5 일 평균 거래대금 조건(대회 유동성) 통과 여부 표시 및 따로 집계
  - 기간 분할: 앞 구간(조정용) / 뒤 구간(검증용) 따로 집계
  - 포트폴리오 시뮬레이션(동시 보유 제한), 랜덤 진입 대조군
"""

import os
import sys
import time
import numpy as np
import pandas as pd

# 이 프로젝트 폴더 = 이 파일이 있는 폴더 (자동)
BASE_DIR = os.path.dirname(os.path.abspath(__file__))   # 이 파일이 있는 폴더 (어디서 실행해도 동일)
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)           # 작업 폴더가 어디든 heikin_scanner_v2.py 를 찾을 수 있게

import heikin_scanner_v2 as hs   # BASE_DIR 안의 heikin_scanner_v2.py (신호 탐지 함수, 설정값 재사용)


def P(filename):
    return os.path.join(BASE_DIR, filename)


# ========================== [설정] ==========================
UNIVERSE_PATH = P('stock_universe.csv')
CACHE_DIR = P('data_cache')
CACHE_MAX_AGE_DAYS = 1      # 캐시 파일이 이 일수보다 오래되면 다시 다운로드
OUT_TRADES = P('trades_v2.csv')
DELISTED_CACHE = P('delisted_codes.csv')   # 상장폐지/소멸 종목 목록 캐시 (지우면 다시 만듦)
EXCLUDE_SPAC = True         # 이름에 '스팩' 들어간 사라진 종목 제외 (합병 전 껍데기 회사)
DELIST_GAP_DAYS = 10        # 데이터가 오늘보다 이만큼 일찍 끝나면 '상장폐지'로 간주

START = '2021-01-01'
END = None                  # None 이면 오늘까지
N_STOCKS = 300              # 처음엔 30 으로 오류 확인 -> 300 -> 전체는 None (시간 오래 걸림, 수십 분~)
SEED = 42

MAX_HOLD = 60               # 최대 보유 거래일 (지나면 마지막 날 종가 청산)
COST_RT = 0.003             # 왕복 거래비용 가정 0.3% (수수료+세금+슬리피지, 직접 확인/조정 필요)
N_RANDOM = 200              # 랜덤 대조군 반복 횟수
WARMUP = 60                 # 랜덤 진입일은 데이터 시작 후 이 봉수 이후에서만 뽑음

SPLIT_DATE = '2024-01-01'   # 이 날짜 이전 신호 = 조정용 구간, 이후 = 검증용 구간
MAX_POS = 10                # 포트폴리오 시뮬: 동시 최대 보유 종목 수 (1 종목 비중 = 1/MAX_POS)
MIN_TRADES_FOR_CONTROL = 20 # 거래가 이보다 적으면 랜덤 대조군은 건너뜀
USE_FLIP_EXIT = True        # 히말라야(스무스 HA 양->음) 전환 시 매도. False 로 하면 손절/익절/기간만료만
# ============================================================

VARIANTS = {
    'N': 'EMA 조건 없음 (3 조건만, 기준선)',
    'R': 'EMA20 완화 (종가가 EMA20 위)',
    'S': 'EMA20 확실 (아래->위 돌파)',
}
EMA_MODE = {'N': 'none', 'R': 'relaxed', 'S': 'strict'}


def _today_str():
```

```
    return pd.Timestamp.today().strftime('%Y%m%d')


def load_prices(code):
    """pykrx 로 일봉(수정주가) 다운로드 + 하루짜리 캐시. 상장폐지 종목도 폐지 전까지 가져옴."""
    os.makedirs(CACHE_DIR, exist_ok=True)
    path = os.path.join(CACHE_DIR, f'{code}.csv')
    fresh = os.path.exists(path) and (time.time() - os.path.getmtime(path)) < CACHE_MAX_AGE_DAYS * 86400
    if fresh:
        df = pd.read_csv(path, index_col=0, parse_dates=True)
    else:
        end = (END or _today_str()).replace('-', '')
        try:
            df = hs.stock.get_market_ohlcv(START.replace('-', ''), end, code)
        except Exception as e:
            print(f'  [다운로드 실패] {code}: {e}')
            return None
        if df is None or len(df) == 0:
            return None
        df = df.rename(columns={'시가': 'open', '고가': 'high', '저가': 'low',
                                '종가': 'close', '거래량': 'volume'})
        df = df[['open', 'high', 'low', 'close', 'volume']]
        df.to_csv(path)
        time.sleep(0.2)
    df = df[['open', 'high', 'low', 'close', 'volume']].astype(float)
    df = df[df['volume'] > 0]
    return df if len(df) >= 150 else None


def _list_tickers(date_str):
    out = []
    for mk in ('KOSPI', 'KOSDAQ'):
        out += hs.stock.get_market_ticker_list(date_str, market=mk)
    return out


def _tickers_near(date, forward=True):
    """date 가 휴장일이면 앞(또는 뒤)으로 하루씩 옮겨 가며 종목 목록이 나오는 날을 찾는다."""
    for k in range(8):
        d = date + pd.Timedelta(days=k if forward else -k)
        if d > pd.Timestamp.today():
            continue
        try:
            t = _list_tickers(d.strftime('%Y%m%d'))
        except Exception:
            t = []
        if t:
            return t
    return []


def get_delisted(universe_codes):
    """
    START 이후 분기마다 상장 종목 목록을 모아서, 지금 목록과 대회 종목 목록에 없는 종목 = 사라진 종목을 찾는다.
    결과는 DELISTED_CACHE 에 저장해 두고 다음부터는 그 파일을 쓴다.
    """
    if os.path.exists(DELISTED_CACHE):
        d = pd.read_csv(DELISTED_CACHE, dtype=str)
        print(f'[상장폐지 종목] 캐시 사용: {len(d)}개 ({DELISTED_CACHE})')
        return d
    print('[상장폐지 종목] 과거 상장목록 조회 중 (처음 한 번만, 몇 분 걸림)...')
    seen = set()
    for d0 in pd.date_range(pd.Timestamp(START), pd.Timestamp.today(), freq='QS'):
        seen.update(_tickers_near(d0, forward=True))
    now = set(_tickers_near(pd.Timestamp.today().normalize(), forward=False))
    gone = sorted(seen - now - set(universe_codes))
    rows = []
    for code in gone:
        try:
            nm = hs.stock.get_market_ticker_name(code)
        except Exception:
            nm = ''
        if not isinstance(nm, str):
            nm = ''
        if EXCLUDE_SPAC and '스팩' in nm:
            continue
        rows.append((code, nm))
    d = pd.DataFrame(rows, columns=['code', '종목명'])
```

```
    d.to_csv(DELISTED_CACHE, index=False, encoding='utf-8-sig')
    print(f'[상장폐지 종목] 사라진 종목 {len(gone)}개 중 스팩 제외 {len(d)}개 저장 -> {DELISTED_CACHE}')
    return d


def prepare(raw):
    """신호 탐지 + 히말라야 전환 + 유동성 판단에 필요한 열을 모두 계산."""
    raw = hs.add_indicators(raw)
    ha1 = hs.heikin_ashi_raw(raw)
    ha2 = hs.smoothed_heikin_ashi(raw, hs.HA_LEN, hs.HA_LEN2)
    df = raw.join(ha1).join(ha2)          # 지표가 아직 계산 안 된 초반 봉은 NaN 그대로 둠 (비교는 False 처리됨)
    bull2 = (df['c2'] > df['o2']).fillna(False)
    bear2 = (df['o2'] > df['c2']).fillna(False)
    df['flip'] = bull2.shift(1, fill_value=False) & bear2     # 스무스 HA 양봉 -> 음봉 전환 봉
    value = df['close'] * df['volume']                          # 거래대금 근사 (종가 x 거래량)
    df['liq_ok'] = value.rolling(hs.TRADING_VALUE_AVG_DAYS).mean() >= hs.MIN_AVG_TRADING_VALUE
    return df


def sim_trade(o, h, l, c, flip, i_sig, stop=None, stop_pct=None, use_flip=False):
    """
    신호일 i_sig 종가 후 -> 다음날 시가 진입.
    청산: 손절 / 익절 / (옵션) 히말라야 전환 다음날 시가 / 기간만료 종가 중 먼저 오는 것.
    같은 날 손절과 익절이 모두 가능하면 손절 먼저 (보수적). 갭으로 시가가 이미 손절가 아래/익절가 위면 시가 청산.
    반환: (진입가, 청산봉 위치, 청산가, 사유, 손절가, 익절가) 또는 None
    """
    n = len(o)
    e = i_sig + 1
    if e >= n:
        return None
    entry = o[e]
    if not entry > 0:
        return None
    if stop is None:
        stop = entry * (1 - stop_pct)
    if stop >= entry:
        return None
    target = entry + hs.RR_RATIO * (entry - stop)

    last = min(n - 1, e + MAX_HOLD - 1)
    oo, hh, ll = o[e:last + 1], h[e:last + 1], l[e:last + 1]
    k = len(oo)
    first = np.arange(k) == 0
    gap_stop = (~first) & (oo <= stop)
    gap_tgt = (~first) & (oo >= target)
    hit_stop = ll <= stop
    hit_tgt = hh >= target
    any_ev = gap_stop | gap_tgt | hit_stop | hit_tgt
    ev = int(np.argmax(any_ev)) if any_ev.any() else None

    fl = None
    if use_flip:
        fz = flip[e:last + 1]
        if fz.any():
            f = int(np.argmax(fz))
            if f + 1 < k:               # 전환 다음날 시가에 청산 (기간 안에 들어올 때만)
                fl = f

    if fl is not None and (ev is None or ev > fl):
        d = e + fl + 1
        return entry, d, o[d], '히말라야', stop, target

    if ev is not None:
        d = e + ev
        if gap_stop[ev]:
            return entry, d, o[d], '손절(갭)', stop, target
        if gap_tgt[ev]:
            return entry, d, o[d], '익절(갭)', stop, target
        if hit_stop[ev]:
            return entry, d, stop, '손절', stop, target
        return entry, d, target, '익절', stop, target

    return entry, last, c[last], '기간만료', stop, target


def run_real(df, code, name, variant, delisted=False):
    sigs = hs.find_signals(df, hs.TREND_WINDOW, EMA_MODE[variant])
    o, h, l, c = (df[k].to_numpy() for k in ('open', 'high', 'low', 'close'))
```

```
    flip = df['flip'].to_numpy()
    liq = df['liq_ok'].to_numpy()
    trades = []
    busy_until = -1
    for sg in sigs:
        i = sg['idx']
        if i + 1 <= busy_until:           # 이미 보유 중이면 새 신호는 건너뜀
            continue
        r = sim_trade(o, h, l, c, flip, i, stop=sg['stop'], use_flip=USE_FLIP_EXIT)
        if r is None:
            continue
        entry, d_exit, px_exit, reason, stop, target = r
        busy_until = d_exit
        if delisted and reason == '기간만료' and d_exit == len(o) - 1:
            reason = '상폐청산'            # 데이터가 끝날 때(상장폐지/합병)까지 들고 있다가 마지막 종가로 정리
        trades.append({
            '전략': variant, '종목코드': code, '종목명': name,
            '신호일': df.index[i].date(), '진입일': df.index[i + 1].date(),
            '청산일': df.index[d_exit].date(),
            '진입가': entry, '손절가': stop, '익절가': target, '청산가': px_exit,
            '청산사유': reason,
            '수익률': px_exit / entry - 1 - COST_RT,
            'R배수': (px_exit - entry) / (entry - stop),
            '보유일': d_exit - (i + 1) + 1,
            '손절폭': (entry - stop) / entry,
            'EMA확실': bool(sg['ema_cross']),
            '유동성통과': bool(liq[i]),
            '사라진종목': bool(delisted),
        })
    return trades


def summarize(t, title):
    if len(t) == 0:
        print(f'\n[{title}] 거래 0건')
        return
    r = t['수익률']
    win = r[r > 0]
    lose = r[r <= 0]
    pf = win.sum() / -lose.sum() if len(lose) and lose.sum() != 0 else float('nan')
    print(f'\n[{title}] 거래 {len(t)}건')
    print(f'  승률 {len(win) / len(t) * 100:.1f}% | 평균수익률 {r.mean() * 100:.2f}% | 중앙값 {r.median() * 100:.2f}%')
    print(f'  평균이익 {win.mean() * 100 if len(win) else 0:.2f}% | 평균손실 {lose.mean() * 100 if len(lose) else 0:.2f}% | Profit
Factor {pf:.2f}')
    print(f'  평균R배수 {t["R배수"].mean():.2f} | 평균 보유 {t["보유일"].mean():.1f}일')
    print('  청산사유별 건수:', t['청산사유'].value_counts().to_dict())


def portfolio_sim(t, title):
    """
    동시 보유 MAX_POS 종목 제한, 종목당 비중 1/MAX_POS 로 거래를 순서대로 받아 수익률 / MDD 를 근사.
    (청산 시점에 실현된 손익만 반영하는 근사치. 같은 날 신호가 넘치면 EMA 확실 돌파를 먼저 채움)
    """
    if len(t) == 0:
        return
    t = t.sort_values(['진입일', 'EMA확실'], ascending=[True, False])
    w = 1.0 / MAX_POS
    open_exits, acc, skipped = [], [], 0
    for r in t.itertuples(index=False):
        open_exits = [x for x in open_exits if x >= r.진입일]
        if len(open_exits) >= MAX_POS:
            skipped += 1
            continue
        acc.append(r)
        open_exits.append(r.청산일)
    acc.sort(key=lambda r: r.청산일)
    eq, curve = 1.0, []
    for r in acc:
        eq *= 1 + w * r.수익률
        curve.append(eq)
    curve = np.array(curve)
    mdd = (curve / np.maximum.accumulate(curve) - 1).min()
    years = max((acc[-1].청산일 - t['진입일'].min()).days / 365.25, 1e-9)
    cagr = eq ** (1 / years) - 1 if years >= 0.5 else float('nan')
    print(f'\n포트폴리오 근사 - {title}] 동시보유 최대 {MAX_POS}종목, 종목당 {w * 100:.0f}%')
    print(f'  체결 {len(acc)}건 / 자리 없어 건너뜀 {skipped}건')
    print(f'  누적수익률 {(eq - 1) * 100:.1f}% | 연환산 {cagr * 100:.1f}% | 최대낙폭(실현 기준 근사) {mdd * 100:.1f}%')
```

```
def random_control(trades_df, store, use_flip, use_liq_pool, title):
    """
    실제 거래 한 건마다 같은 종목의 무작위 날짜에 같은 손절폭 / 같은 손익비 / 같은 청산규칙으로 진입한 가상 거래를 만들고,
    실제 거래 수만큼의 평균수익률을 N_RANDOM 번 반복해 '운으로 얻을 평균수익률 분포'와 비교한다.
    use_liq_pool=True 면 무작위 날짜도 유동성 조건을 통과한 날 중에서만 뽑는다.
    """
    if len(trades_df) < MIN_TRADES_FOR_CONTROL:
        print(f'\n[랜덤 대조군 - {title}] 거래 {len(trades_df)}건 < {MIN_TRADES_FOR_CONTROL}건이라 건너뜀')
        return
    rng = np.random.default_rng(SEED)
    T = len(trades_df)
    rets = np.full((T, N_RANDOM), np.nan)
    for t_i, row in enumerate(trades_df.itertuples(index=False)):
        o, h, l, c, flip, liq = store[row.종목코드]
        n = len(o)
        ok = np.arange(n)
        mask = (ok >= WARMUP) & (ok <= n - 2)     # 실제 신호와 똑같이 데이터 끝(상장폐지 포함)에서 잘리는 것도 허용
        if use_liq_pool:
            mask &= liq
        pool = ok[mask]
        if len(pool) == 0:
            continue
        days = rng.choice(pool, size=N_RANDOM)
        for k, d in enumerate(days):
            r = sim_trade(o, h, l, c, flip, int(d), stop_pct=row.손절폭, use_flip=use_flip)
            if r is None:
                continue
            rets[t_i, k] = r[2] / r[0] - 1 - COST_RT
    null_means = np.nanmean(rets, axis=0)
    real_mean = trades_df['수익률'].mean()
    p = (np.sum(null_means >= real_mean) + 1) / (N_RANDOM + 1)
    print(f'\n[랜덤 대조군 - {title}]')
    print(f'  실제 신호 평균수익률 {real_mean * 100:.2f}%')
    print(f'  랜덤 진입 평균수익률 분포: 평균 {np.nanmean(null_means) * 100:.2f}% | 5~95% 구간 '
          f'{np.nanpercentile(null_means, 5) * 100:.2f}% ~ {np.nanpercentile(null_means, 95) * 100:.2f}%')
    print(f'  p값(랜덤이 실제 이상일 비율) {p:.3f}  (작을수록 운이 아닐 가능성이 큼, 0.05 이하면 유의)')


def report(t, variant):
    name = f'기준 {variant}: {VARIANTS[variant]}'
    print('\n' + '=' * 70)
    print(name)
    print('=' * 70)
    if len(t) == 0:
        print('거래 없음')
        return
    summarize(t, f'{variant} | 전체 (유동성 무관)')
    summarize(t[t['유동성통과']], f'{variant} | 유동성 통과(5일 평균 거래대금 조건)만')
    summarize(t[t['사라진종목']], f'{variant} | 그중 사라진(상장폐지/합병) 종목 거래만')

    split = pd.Timestamp(SPLIT_DATE).date()
    summarize(t[t['신호일'] < split], f'{variant} | 조정용 구간 (신호일 < {SPLIT_DATE})')
    summarize(t[t['신호일'] >= split], f'{variant} | 검증용 구간 (신호일 >= {SPLIT_DATE})')


    portfolio_sim(t[t['유동성통과']], f'{variant} | 유동성 통과 거래만')


def main():
    if not os.path.isdir(BASE_DIR):
        print(f'[오류] BASE_DIR 폴더를 찾을 수 없어요: {BASE_DIR}\n  코드 위쪽 BASE_DIR 경로를 확인하세요.')
        return
    uni = pd.read_csv(UNIVERSE_PATH, dtype=str)
    uni['code'] = uni['종목코드'].str.replace('^A', '', regex=True)
    base = pd.DataFrame({'code': uni['code'], 'name': uni['종목명'], 'gone': False})

    gone = get_delisted(set(uni['code']))
    gone_df = pd.DataFrame({'code': gone['code'], 'name': gone['종목명'], 'gone': True})
    allu = pd.concat([base, gone_df], ignore_index=True).drop_duplicates('code')
    print(f'백테스트 대상: 현재 종목 {len(base)}개 + 사라진 종목 {int(allu["gone"].sum())}개')

    if N_STOCKS is not None:
        allu = allu.sample(min(N_STOCKS, len(allu)), random_state=SEED)
        print(f'  무작위 {len(allu)}개 사용 (그중 사라진 종목 {int(allu["gone"].sum())}개)')

    cutoff = pd.Timestamp.today().normalize() - pd.Timedelta(days=DELIST_GAP_DAYS)
    trades = {v: [] for v in VARIANTS}
```

```
    store = {}
    for k, row in enumerate(allu.itertuples(index=False), 1):
        code, name = row.code, row.name
        raw = load_prices(code)
        if raw is None:
            continue
        delisted = bool(raw.index[-1] < cutoff)
        try:
            df = prepare(raw)
        except Exception as e:
            print(f'  [계산 실패] {name}({code}): {e}')
            continue
        store[code] = tuple(df[x].to_numpy() for x in ('open', 'high', 'low', 'close')) + (
            df['flip'].to_numpy(), df['liq_ok'].to_numpy())
        for v in VARIANTS:
            trades[v].extend(run_real(df, code, name, v, delisted))
        if k % 20 == 0 or k == len(allu):
            print(f'  진행 {k}/{len(allu)}')

    dfs = {}
    for v in VARIANTS:
        dfs[v] = pd.DataFrame(trades[v])
        if len(dfs[v]):
            dfs[v] = dfs[v].sort_values('진입일').reset_index(drop=True)

    if all(len(d) == 0 for d in dfs.values()):
        print('거래가 하나도 없습니다. 종목 수를 늘리거나 기간을 확인하세요.')
        return

    all_t = pd.concat([d for d in dfs.values() if len(d)], ignore_index=True)
    all_t.to_csv(OUT_TRADES, index=False, encoding='utf-8-sig')
    print(f'\n{OUT_TRADES} 저장 (N+R+S 합계 {len(all_t)}건, 같은 거래가 기준별로 겹쳐 들어있으니 전략 열로 구분)')

    for v in VARIANTS:
        report(dfs[v], v)
        if len(dfs[v]):
            use_flip = USE_FLIP_EXIT
            random_control(dfs[v], store, use_flip, False, f'{v} | 전체')
            random_control(dfs[v][dfs[v]['유동성통과']], store, use_flip, True, f'{v} | 유동성 통과')


if __name__ == '__main__':
    main()
```

C-3. backtest_sweep_v2.py (스윕 백테스트: 손절폭 × 손익비 × 코스피 필터 72 개 조합)

```
# -*- coding: utf-8 -*-
"""
하이킨아시 v2 스윕 백테스트: 손익비(RR) x 시장 국면 필터 비교
================================================================
backtest_heikin_v2.py 와 heikin_scanner_v2.py 를 그대로 가져다 쓴다 (두 파일은 수정하지 않음).
같은 폴더(BASE_DIR)에 이 파일을 backtest_sweep_v2.py 로 저장하고 실행.

[비교하는 것]
  1) 손절폭 배수: 1.0(기존) / 2.0(손절폭 2 배)
       새 손절가 = 진입가 - 배수 x (진입가 - 기존 손절가)
  2) 손익비(RR): 0.5 / 1.0 / 1.5 / 2.0   (새 손절폭 기준. 익절가 = 진입가 + RR x (진입가 - 새 손절가))
  3) 시장 국면 필터 (신호일 종가 기준, 코스피 지수):
       없음 / 코스피 > 20 일 이동평균 / 코스피 > 60 일 이동평균
  4) EMA 기준 3 종 (N 없음 / R 완화 / S 확실)

[핵심: 랜덤 대조군]
  손익비를 낮추면 승률은 당연히 올라가지만, 랜덤 진입도 똑같이 올라간다.
  그래서 각 조합마다 "같은 국면 / 같은 손익비 / 같은 청산규칙"의 랜덤 진입과 비교한 p 값을 같이 본다.
  (유동성 통과 신호만 평가: 대회에서 실제로 살 수 있는 종목)
"""

import os
import sys
import time
import numpy as np
import pandas as pd

# 이 프로젝트 폴더 = 이 파일이 있는 폴더 (자동). 모든 .py 파일을 같은 폴더에 둘 것
BASE_DIR = os.path.dirname(os.path.abspath(__file__))   # 이 파일이 있는 폴더 (어디서 실행해도 동일)
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)

import heikin_scanner_v2 as hs
import backtest_heikin_v2 as bt


def P(filename):
    return os.path.join(BASE_DIR, filename)


# ========================== [설정] ==========================
RR_LIST = [0.5, 1.0, 1.5, 2.0]                    # 비교할 손익비
REGIMES = {'없음': None, '코스피>MA20': 20, '코스피>MA60': 60}   # 시장 국면 필터 (이동평균 일수)
VARIANT_LIST = ['N', 'R', 'S']                     # 비교할 EMA 기준 (bt.VARIANTS 의 키)
STOP_MULTS = [1.0, 2.0]                            # 손절폭 배수. 1.0 = 기존(도지 저가 기준), 2.0 = 손절폭을 2 배로 넓힘

N_STOCKS = bt.N_STOCKS        # backtest_heikin_v2.py 의 값을 따라감 (300 또는 None). 여기서 바로 바꿔도 됨
RANDOM_CONTROL = True         # 랜덤 대조군 계산 여부 (False 면 훨씬 빠르지만 p 값이 없음)
N_RANDOM_SWEEP = 100          # 조합마다 랜덤 반복 횟수 (조합이 많아서 bt.N_RANDOM 보다 적게)
INDEX_CACHE = P('index_kospi.csv')
OUT_SUMMARY = P('sweep_summary_v2.csv')
OUT_TRADES = P('sweep_trades_v2.csv')
# ============================================================


def load_kospi():
    """코스피 지수 일봉(종가) 가져오기. pykrx 먼저, 안 되면 FinanceDataReader. 하루짜리 캐시."""
    fresh = os.path.exists(INDEX_CACHE) and (time.time() - os.path.getmtime(INDEX_CACHE)) < 86400
    if fresh:
        s = pd.read_csv(INDEX_CACHE, index_col=0, parse_dates=True)['close']
        return s.astype(float)
    start = (pd.Timestamp(bt.START) - pd.Timedelta(days=200)).strftime('%Y%m%d')   # 이동평균 계산용 여유
    end = pd.Timestamp.today().strftime('%Y%m%d')
    s = None
    try:
        d = hs.stock.get_index_ohlcv(start, end, '1001')       # 1001 = 코스피
        if d is not None and len(d) > 0:
            s = d['종가'].astype(float)
    except Exception as e:
        print(f'  [pykrx 지수 실패] {e}')
    if s is None or len(s) == 0:
        try:
            import FinanceDataReader as fdr
            d = fdr.DataReader('KS11', pd.Timestamp(start).strftime('%Y-%m-%d'))
```

```
            s = d['Close'].astype(float)
        except Exception as e:
            print(f'  [FinanceDataReader 지수 실패] {e}')
            return None
    s.index = pd.to_datetime(s.index)
    s.name = 'close'
    s.to_frame().to_csv(INDEX_CACHE)
    return s


def regime_series(idx_close):
    """REGIMES 각각에 대해 '신호일 종가가 이동평균 위인가' 불리언 시리즈."""
    out = {}
    for name, n in REGIMES.items():
        if n is None:
            out[name] = None
        else:
            out[name] = (idx_close > idx_close.rolling(n).mean())
    return out


def build_universe():
    uni = pd.read_csv(bt.UNIVERSE_PATH, dtype=str)
    uni['code'] = uni['종목코드'].str.replace('^A', '', regex=True)
    base = pd.DataFrame({'code': uni['code'], 'name': uni['종목명'], 'gone': False})
    gone = bt.get_delisted(set(uni['code']))
    gone_df = pd.DataFrame({'code': gone['code'], 'name': gone['종목명'], 'gone': True})
    allu = pd.concat([base, gone_df], ignore_index=True).drop_duplicates('code')
    if N_STOCKS is not None:
        allu = allu.sample(min(N_STOCKS, len(allu)), random_state=bt.SEED)
    return allu


def load_all(allu, reg_series):
    """종목마다 가격 / 신호 / 국면 배열을 한 번만 계산해서 store 에 보관."""
    cutoff = pd.Timestamp.today().normalize() - pd.Timedelta(days=bt.DELIST_GAP_DAYS)
    store = {}
    for k, row in enumerate(allu.itertuples(index=False), 1):
        raw = bt.load_prices(row.code)
        if raw is None:
            continue
        try:
            df = bt.prepare(raw)
        except Exception as e:
            print(f'  [계산 실패] {row.name}({row.code}): {e}')
            continue
        regs = {}
        for rn, rs in reg_series.items():
            if rs is None:
                regs[rn] = None
            else:
                regs[rn] = rs.reindex(df.index, method='ffill').fillna(False).to_numpy()
        sigs = {v: hs.find_signals(df, hs.TREND_WINDOW, bt.EMA_MODE[v]) for v in VARIANT_LIST}
        store[row.code] = {
            'name': row.name,
            'gone': bool(raw.index[-1] < cutoff),
            'o': df['open'].to_numpy(), 'h': df['high'].to_numpy(),
            'l': df['low'].to_numpy(), 'c': df['close'].to_numpy(),
            'flip': df['flip'].to_numpy(), 'liq': df['liq_ok'].to_numpy(),
            'dates': df.index, 'regs': regs, 'sigs': sigs,
        }
        if k % 50 == 0 or k == len(allu):
            print(f'  데이터 준비 {k}/{len(allu)}')
    return store


def run_combo(store, variant, rr, reg_name, mult):
    """한 조합(EMA 기준, 손익비, 국면 필터, 손절폭 배수)의 실제 거래 목록."""
    hs.RR_RATIO = rr                      # bt.sim_trade 가 이 값으로 익절가를 계산
    rows = []
    for code, d in store.items():
        o, h, l, c, flip, liq = d['o'], d['h'], d['l'], d['c'], d['flip'], d['liq']
        reg = d['regs'][reg_name]
        busy_until = -1
        for sg in d['sigs'][variant]:
            i = sg['idx']
            if reg is not None and not reg[i]:      # 국면 필터에 걸린 신호는 거래 자체를 안 함
                continue
```

```
            if i + 1 <= busy_until:                # 보유 중이면 새 신호 건너뜀
                continue
            if i + 1 >= len(o):
                continue
            ent = o[i + 1]                         # 진입가 = 다음날 시가
            new_stop = ent - mult * (ent - sg['stop'])      # 손절폭을 mult 배로
            if not (0 < new_stop < ent):
                continue
            r = bt.sim_trade(o, h, l, c, flip, i, stop=new_stop, use_flip=bt.USE_FLIP_EXIT)
            if r is None:
                continue
            entry, d_exit, px_exit, reason, stop, target = r
            busy_until = d_exit
            rows.append({
                '변형': variant, '국면': reg_name, '손절배수': mult, 'RR': rr, '종목코드': code, '종목명': d['name'],
                '신호일': d['dates'][i], '청산사유': reason,
                '수익률': px_exit / entry - 1 - bt.COST_RT,
                '손절폭': (entry - stop) / entry,
                '유동성통과': bool(liq[i]),
            })
    return pd.DataFrame(rows)


_pool_cache = {}


def _pool(store, code, reg_name):
    key = (code, reg_name)
    if key not in _pool_cache:
        d = store[code]
        n = len(d['o'])
        ok = np.arange(n)
        mask = (ok >= bt.WARMUP) & (ok <= n - 2) & d['liq']
        reg = d['regs'][reg_name]
        if reg is not None:
            mask &= reg                        # 랜덤도 같은 국면의 날에서만 뽑는다
        _pool_cache[key] = ok[mask]
    return _pool_cache[key]


def random_p(store, t_liq, rr, reg_name):
    """같은 국면 / 같은 손익비 / 같은 청산규칙의 랜덤 진입과 비교. 반환: (랜덤 평균, p 값)."""
    if len(t_liq) < bt.MIN_TRADES_FOR_CONTROL:
        return float('nan'), float('nan')
    hs.RR_RATIO = rr
    rng = np.random.default_rng(bt.SEED)
    T = len(t_liq)
    rets = np.full((T, N_RANDOM_SWEEP), np.nan)
    for t_i, row in enumerate(t_liq.itertuples(index=False)):
        d = store[row.종목코드]
        pool = _pool(store, row.종목코드, reg_name)
        if len(pool) == 0:
            continue
        days = rng.choice(pool, size=N_RANDOM_SWEEP)
        for k, day in enumerate(days):
            r = bt.sim_trade(d['o'], d['h'], d['l'], d['c'], d['flip'], int(day),
                             stop_pct=row.손절폭, use_flip=bt.USE_FLIP_EXIT)
            if r is None:
                continue
            rets[t_i, k] = r[2] / r[0] - 1 - bt.COST_RT
    null_means = np.nanmean(rets, axis=0)
    real = t_liq['수익률'].mean()
    p = (np.sum(null_means >= real) + 1) / (N_RANDOM_SWEEP + 1)
    return float(np.nanmean(null_means)), float(p)


def stats(t):
    r = t['수익률']
    win = r[r > 0]
    lose = r[r <= 0]
    pf = win.sum() / -lose.sum() if len(lose) and lose.sum() != 0 else float('nan')
    return len(t), (len(win) / len(t) * 100 if len(t) else float('nan')), r.mean() * 100, pf


def main():
    if not os.path.isdir(BASE_DIR):
        print(f'[오류] BASE_DIR 폴더를 찾을 수 없어요: {BASE_DIR}')
        return
```

```
    print('[1/4] 코스피 지수 불러오는 중...')
    idx_close = load_kospi()
    if idx_close is None:
        print('  코스피 지수를 못 가져와서 시장 국면 필터 없이 손익비만 비교합니다.')
        regimes_all = {'없음': None}
    else:
        regimes_all = regime_series(idx_close)
        print(f'  코스피 {len(idx_close)}일치 ({idx_close.index[0].date()} ~ {idx_close.index[-1].date()})')
        for rn, rs in regimes_all.items():
            if rs is not None:
                print(f'  {rn}: 전체 기간 중 {rs.mean() * 100:.0f}% 날짜가 조건 충족')
    global REGIMES
    REGIMES = {k: v for k, v in REGIMES.items() if k in regimes_all}

    print('[2/4] 종목 데이터 / 신호 준비...')
    allu = build_universe()
    print(f'  대상 {len(allu)}개 (사라진 종목 {int(allu["gone"].sum())}개)')
    store = load_all(allu, regimes_all)

    print('[3/4] 조합별 백테스트...')
    split = pd.Timestamp(bt.SPLIT_DATE)
    summary, all_trades = [], []
    combos = [(v, rn, m, rr) for v in VARIANT_LIST for rn in REGIMES for m in STOP_MULTS for rr in RR_LIST]
    for n_c, (v, rn, m, rr) in enumerate(combos, 1):
        t = run_combo(store, v, rr, rn, m)
        t_liq = t[t['유동성통과']] if len(t) else t
        if len(t_liq) == 0:
            continue
        n, wr, avg, pf = stats(t_liq)
        is_avg = t_liq.loc[t_liq['신호일'] < split, '수익률'].mean() * 100
        oos_avg = t_liq.loc[t_liq['신호일'] >= split, '수익률'].mean() * 100
        if RANDOM_CONTROL:
            rnd_mean, p = random_p(store, t_liq, rr, rn)
        else:
            rnd_mean, p = float('nan'), float('nan')
        summary.append({
            '변형': v, '국면': rn, '손절배수': m, 'RR': rr, '평균손절폭%': round(t_liq['손절폭'].mean() * 100, 1), '거래수': n, '승률
%': round(wr, 1),
            '평균수익%': round(avg, 2), 'PF': round(pf, 2),
            '조정구간%': round(is_avg, 2), '검증구간%': round(oos_avg, 2),
            '랜덤평균%': round(rnd_mean * 100, 2) if rnd_mean == rnd_mean else float('nan'),
            '신호-랜덤%p': round(avg - rnd_mean * 100, 2) if rnd_mean == rnd_mean else float('nan'),
            'p값': round(p, 3) if p == p else float('nan'),
        })
        all_trades.append(t_liq)
        print(f'  {n_c}/{len(combos)} 완료 [{v} | {rn} | 손절x{m} | RR {rr}] 거래 {n}건 평균 {avg:.2f}%')

    print('[4/4] 결과 정리...')
    sm = pd.DataFrame(summary)
    sm.to_csv(OUT_SUMMARY, index=False, encoding='utf-8-sig')
    if all_trades:
        pd.concat(all_trades, ignore_index=True).to_csv(OUT_TRADES, index=False, encoding='utf-8-sig')

    pd.set_option('display.width', 200)
    pd.set_option('display.max_rows', 500)
    pd.set_option('display.unicode.east_asian_width', True)
    print('\n' + '=' * 100)
    print('결과 (유동성 통과 신호만 / 거래비용 0.3% 반영 / 히말라야 매도 포함)')
    print('  신호-랜덤%p = 평균수익% - 랜덤평균%  (같은 국면, 같은 RR 기준. 양수일수록 패턴이 랜덤보다 낫다는 뜻)')
    print('  p값 = 랜덤이 실제 이상일 비율 (0.05 이하면 유의)')
    print('=' * 100)
    for v in VARIANT_LIST:
        print(f'\n--- {v}: {bt.VARIANTS[v]} ---')
        print(sm[sm['변형'] == v].drop(columns='변형').to_string(index=False))

    n_tests = len(sm)
    print(f'\n주의: {n_tests}개 조합을 한꺼번에 비교했으므로 p값 0.05 이하가 몇 개 나와도 일부는 우연입니다.')
    print(f'      (우연만으로도 평균 {n_tests * 0.05:.1f}개는 0.05 이하로 나옴). 조정구간/검증구간 둘 다 좋은 조합만 의심해 볼 것.')
    print(f'\n저장: {OUT_SUMMARY}')


if __name__ == '__main__':
    main()
```

C-4. backtest_forward_v2.py (고정 보유기간 테스트: 5 / 10 / 20 일, 같은 종목 랜덤 비교)

```
# -*- coding: utf-8 -*-
"""
하이킨아시 v2 고정 보유기간 테스트 (손절/익절/히말라야 없음)
================================================================
질문: "신호가 뜬 종목은 N 일 뒤에 같은 종목의 아무 날 대비 더 올라 있었나?"
  - 손절, 익절, 히말라야 청산을 전부 뺀 순수한 패턴 효과를 본다.
  - 신호일 종가 확인 -> 다음날 시가 진입 -> H 거래일 보유 후 종가 청산, 거래비용(왕복) 차감.
  - 비교 대상(랜덤): 같은 종목, 같은 국면(코스피 필터 있을 때), 유동성 통과한 무작위 날에 똑같이 진입.

backtest_sweep_v2.py, backtest_heikin_v2.py, heikin_scanner_v2.py 와 같은 폴더에서 실행 (모두 수정하지 않음).

[읽는 법]
  평균수익%      : 신호 진입의 평균 H 일 수익률 (비용 차감 후)
  랜덤평균%      : 같은 종목 무작위 날 진입의 평균
  초과%p         : 신호 - 랜덤 (이게 양수이고 t 값이 2 이상이면 패턴 효과 후보)
  t 값     : 신호별로 (자기 수익률 - 자기 종목 랜덤 평균)을 구해 평균/표준오차. 약 2 이상이면 우연 가능성 낮음
  p 값     : 랜덤을 N_RANDOM 번 반복했을 때 랜덤 평균이 신호 평균 이상인 비율
  조정/검증구간 : 신호일 기준 SPLIT_DATE 전/후 초과%p (둘 다 양수여야 믿을 만함)
"""

import os
import sys
import numpy as np
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))   # 이 파일이 있는 폴더 (어디서 실행해도 동일)
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)

import heikin_scanner_v2 as hs
import backtest_heikin_v2 as bt
import backtest_sweep_v2 as sw


def P(filename):
    return os.path.join(BASE_DIR, filename)


# ========================== [설정] ==========================
HORIZONS = [5, 10, 20]            # 보유 거래일 수
MIN_GAP_DAYS = 20                 # 같은 종목에서 이 거래일 이내에 또 뜬 신호는 같은 신호로 보고 건너뜀 (겹침 방지)
N_STOCKS = bt.N_STOCKS            # 300 또는 None(전체). 여기서 바로 바꿔도 됨
N_RANDOM = 200                    # 랜덤 반복 횟수
LIQ_ONLY = True                   # True: 유동성 통과 신호/랜덤만 (대회에서 실제로 살 수 있는 종목)
OUT_SUMMARY = P('forward_summary_v2.csv')
# ============================================================


def collect_signals(store, variant, reg_name):
    """종목별 신호 위치(국면/유동성/최소 간격 적용)를 모아서 반환: [(code, i), ...]"""
    out = []
    for code, d in store.items():
        reg = d['regs'][reg_name]
        last = -10 ** 9
        for sg in d['sigs'][variant]:
            i = sg['idx']
            if reg is not None and not reg[i]:
                continue
            if LIQ_ONLY and not d['liq'][i]:
                continue
            if i - last < MIN_GAP_DAYS:
                continue
            out.append((code, i))
            last = i
    return out


def fwd_return(d, i, H):
    """신호 위치 i -> 다음날 시가 진입, H 봉째 종가 청산. 데이터가 모자라면 사라진 종목만 마지막 종가로 정리, 나머지는 None."""
    o, c = d['o'], d['c']
    n = len(o)
    if i + 1 >= n or not o[i + 1] > 0:
        return None
    ex = i + H
```

```
    if ex > n - 1:
        if d['gone']:
            ex = n - 1
        else:
            return None
    return c[ex] / o[i + 1] - 1 - bt.COST_RT


def rand_pool(store, code, reg_name, H):
    d = store[code]
    n = len(d['o'])
    ok = np.arange(n)
    mask = (ok >= bt.WARMUP) & (ok <= n - 2)
    if not d['gone']:
        mask &= (ok <= n - 1 - H)         # 정상 종목은 H일 뒤 종가가 존재해야 함
    if LIQ_ONLY:
        mask &= d['liq']
    reg = d['regs'][reg_name]
    if reg is not None:
        mask &= reg
    return ok[mask]


def evaluate(store, sigs, variant, reg_name, H, rng):
    rows = []
    for code, i in sigs:
        r = fwd_return(store[code], i, H)
        if r is None:
            continue
        pool = rand_pool(store, code, reg_name, H)
        if len(pool) == 0:
            continue
        days = rng.choice(pool, size=N_RANDOM)
        d = store[code]
        n = len(d['o'])
        ex = np.minimum(days + H, n - 1)
        rr = d['c'][ex] / d['o'][days + 1] - 1 - bt.COST_RT       # 랜덤 N_RANDOM개 수익률
        rows.append((d['dates'][i], r, rr))
    if not rows:
        return None
    dates = pd.to_datetime([x[0] for x in rows])
    real = np.array([x[1] for x in rows])
    rand = np.vstack([x[2] for x in rows])                  # (신호수, N_RANDOM)
    null_means = rand.mean(axis=0)
    rand_per_sig = rand.mean(axis=1)
    diff = real - rand_per_sig
    n = len(real)
    t_val = diff.mean() / (diff.std(ddof=1) / np.sqrt(n)) if n > 2 and diff.std(ddof=1) > 0 else float('nan')
    p = (np.sum(null_means >= real.mean()) + 1) / (N_RANDOM + 1)
    split = pd.Timestamp(bt.SPLIT_DATE)
    a, b = dates < split, dates >= split
    return {
        '변형': variant, '국면': reg_name, '보유일': H, '신호수': n,
        '평균수익%': round(real.mean() * 100, 2),
        '중앙값%': round(np.median(real) * 100, 2),
        '상승비율%': round((real > 0).mean() * 100, 1),
        '랜덤평균%': round(rand_per_sig.mean() * 100, 2),
        '초과%p': round(diff.mean() * 100, 2),
        't값': round(float(t_val), 2),
        'p값': round(float(p), 3),
        '조정구간초과%p': round(diff[a].mean() * 100, 2) if a.any() else float('nan'),
        '검증구간초과%p': round(diff[b].mean() * 100, 2) if b.any() else float('nan'),
    }


def main():
    if not os.path.isdir(BASE_DIR):
        print(f'[오류] BASE_DIR 폴더를 찾을 수 없어요: {BASE_DIR}')
        return
    print('[1/3] 코스피 지수 / 종목 데이터 준비...')
    idx_close = sw.load_kospi()
    if idx_close is None:
        print('  코스피 지수를 못 가져와서 국면 필터 없이 진행합니다.')
        regimes_all = {'없음': None}
    else:
        regimes_all = sw.regime_series(idx_close)
    sw.N_STOCKS = N_STOCKS
    allu = sw.build_universe()
```

```
    print(f'  대상 {len(allu)}개 (사라진 종목 {int(allu["gone"].sum())}개)')
    store = sw.load_all(allu, regimes_all)

    print('[2/3] 고정 보유기간 테스트...')
    rng = np.random.default_rng(bt.SEED)
    out = []
    for v in sw.VARIANT_LIST:
        for rn in regimes_all:
            sigs = collect_signals(store, v, rn)
            for H in HORIZONS:
                r = evaluate(store, sigs, v, rn, H, rng)
                if r is not None:
                    out.append(r)
            print(f'  완료 [{v} | {rn}] 신호 {len(sigs)}건')

    print('[3/3] 결과 정리...')
    sm = pd.DataFrame(out)
    sm.to_csv(OUT_SUMMARY, index=False, encoding='utf-8-sig')
    pd.set_option('display.width', 220)
    pd.set_option('display.max_rows', 500)
    pd.set_option('display.unicode.east_asian_width', True)
    print('\n' + '=' * 100)
    print(f'고정 보유기간 결과 (손절/익절 없음, 비용 {bt.COST_RT * 100:.1f}% 반영, 유동성 통과만={LIQ_ONLY})')
    print('  초과%p = 신호 평균 - 같은 종목 랜덤 평균 / t 값 2 이상 + 조정/검증 둘 다 양수여야 의미 있음')
    print('=' * 100)
    for v in sw.VARIANT_LIST:
        print(f'\n--- {v}: {bt.VARIANTS[v]} ---')
        print(sm[sm['변형'] == v].drop(columns='변형').to_string(index=False))
    n_tests = len(sm)
    print(f'\n주의: {n_tests}개 조합을 비교했으므로 t 값이 2 를 넘는 것이 한두 개 나와도 우연일 수 있습니다.')
    print(f'      같은 신호가 보유기간(5/10/20 일)과 국면 사이에 겹쳐 있어 조합들은 서로 독립이 아닙니다.')
    print(f'\n저장: {OUT_SUMMARY}')


if __name__ == '__main__':
    main()
```

C-5. screener_plus_v2.py (후보 보조정보: 거래대금, 손절폭, 권장비중 등)

```
# -*- coding: utf-8 -*-
"""
스크리너 보조정보 추가 (screener_plus_v2.py)
============================================================
백테스트 결과상 "신호가 떴다 = 사면 돈 번다"는 근거가 없었으므로,
스캐너가 뽑은 후보를 사람이 판단하기 쉽게 정리해 주는 도구로 쓴다.

입력 : signals_result_v2.xlsx (heikin_scanner_v2.py 가 만든 결과, 같은 폴더)
출력 : signals_enriched_v2.xlsx

각 후보에 붙는 정보
  - 코스피 국면: 코스피 종가가 20일선 / 60일선 위인지
  - 손절폭%: (진입가-손절가)/진입가 -> 작을수록 위험 대비 효율이 좋음
  - 권장비중%: 한 번 손절 났을 때 계좌에서 잃는 비율을 RISK_PER_TRADE_PCT 로 맞추는 비중 (MAX_WEIGHT_PCT 상한)
  - 5일평균거래대금(억), 20일수익률%, 60일고점대비%, 거래량배수(신호일 거래량 / 20일 평균)
  - 정렬: EMA20 확실돌파 먼저 -> 손절폭 작은 순 -> 거래대금 큰 순

heikin_scanner_v2.py 와 같은 폴더(BASE_DIR)에서 실행. 스캐너를 먼저 돌려서 signals_result_v2.xlsx 가 있어야 함.
"""

import os
import sys
import time
import numpy as np
import pandas as pd
from datetime import datetime, timedelta

BASE_DIR = os.path.dirname(os.path.abspath(__file__))   # 이 파일이 있는 폴더 (어디서 실행해도 동일)
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)

import heikin_scanner_v2 as hs


def P(filename):
    return os.path.join(BASE_DIR, filename)


# ============================ [설정] ============================
INPUT_PATH = P('signals_result_v2.xlsx')
OUTPUT_PATH = P('signals_enriched_v2.xlsx')
SHEETS = ['신규신호(일봉)', '신규신호(주봉)']   # 보조정보를 붙일 시트
RISK_PER_TRADE_PCT = 1.0     # 손절 났을 때 계좌에서 잃고 싶은 비율(%). 손절폭이 8%면 권장비중 = 1/8 = 12.5% -> 상한 적용
MAX_WEIGHT_PCT = 10.0        # 한 종목 비중 상한(%)
LOOKBACK_DAYS = 400          # 보조정보 계산용 데이터 기간 (달력일)
SLEEP_SEC = 0.15
# ================================================================


def kospi_regime():
    """코스피 종가가 20일선 / 60일선 위인지. 못 가져오면 (None, None, None)."""
    start = (datetime.today() - timedelta(days=200)).strftime('%Y%m%d')
    end = datetime.today().strftime('%Y%m%d')
    s = None
    try:
        d = hs.stock.get_index_ohlcv(start, end, '1001')
        if d is not None and len(d) > 0:
            s = d['종가'].astype(float)
    except Exception as e:
        print(f'  [pykrx 지수 실패] {e}')
    if s is None:
        try:
            import FinanceDataReader as fdr
            d = fdr.DataReader('KS11', (datetime.today() - timedelta(days=200)).strftime('%Y-%m-%d'))
            s = d['Close'].astype(float)
        except Exception as e:
            print(f'  [FinanceDataReader 지수 실패] {e}')
            return None, None, None
    last = s.iloc[-1]
    ma20 = s.rolling(20).mean().iloc[-1]
    ma60 = s.rolling(60).mean().iloc[-1]
    return last, bool(last > ma20), bool(last > ma60)
```

```
def stock_stats(code, start, end):
    """한 종목의 보조 지표 dict. 데이터가 모자라면 값은 None."""
    out = {'5일평균거래대금(억)': None, '20일수익률%': None, '60일고점대비%': None, '거래량배수': None}
    df = hs.fetch_ohlcv(code, start, end)
    if df is None or len(df) < 25:
        return out
    c, v = df['close'].astype(float), df['volume'].astype(float)
    out['5일평균거래대금(억)'] = round((c * v).tail(5).mean() / 1e8, 1)
    out['20일수익률%'] = round((c.iloc[-1] / c.iloc[-21] - 1) * 100, 1) if len(c) > 21 else None
    out['60일고점대비%'] = round((c.iloc[-1] / c.tail(60).max() - 1) * 100, 1)
    avg20 = v.iloc[-21:-1].mean() if len(v) > 21 else np.nan
    out['거래량배수'] = round(v.iloc[-1] / avg20, 2) if avg20 and avg20 > 0 else None
    return out


def enrich(df, start, end, regime):
    if df.empty:
        return df
    df = df.copy()
    df['종목코드'] = df['종목코드'].astype(str).str.replace('^A', '', regex=True).str.zfill(6)
    last, above20, above60 = regime
    df['코스피>20일선'] = '-' if above20 is None else ('O' if above20 else 'X')
    df['코스피>60일선'] = '-' if above60 is None else ('O' if above60 else 'X')

    stop_pct = (df['진입가'] - df['손절가']) / df['진입가'] * 100
    df['손절폭%'] = stop_pct.round(1)
    df['권장비중%'] = np.minimum(MAX_WEIGHT_PCT, RISK_PER_TRADE_PCT / stop_pct * 100).round(1)

    cache = {}
    extra = []
    for k, code in enumerate(df['종목코드'], 1):
        if code not in cache:
            cache[code] = stock_stats(code, start, end)
            time.sleep(SLEEP_SEC)
        extra.append(cache[code])
        if k % 20 == 0:
            print(f'    보조정보 {k}/{len(df)}')
    df = pd.concat([df.reset_index(drop=True), pd.DataFrame(extra)], axis=1)

    df['_e'] = (df['EMA20확실돌파'] != 'O').astype(int)
    df = df.sort_values(['_e', '손절폭%', '5일평균거래대금(억)'], ascending=[True, True, False]).drop(columns='_e')
    return df.reset_index(drop=True)


def main():
    if not os.path.isfile(INPUT_PATH):
        print(f'[오류] {INPUT_PATH} 가 없어요. heikin_scanner_v2.py 를 먼저 실행하세요.')
        return
    start = (datetime.today() - timedelta(days=LOOKBACK_DAYS)).strftime('%Y%m%d')
    end = datetime.today().strftime('%Y%m%d')

    print('[1/3] 코스피 국면 확인...')
    regime = kospi_regime()
    if regime[1] is None:
        print('  코스피 지수를 못 가져와서 국면 열은 "-" 로 표시합니다.')
    else:
        print(f'  코스피 {regime[0]:.0f} | 20일선 위: {regime[1]} | 60일선 위: {regime[2]}')

    print('[2/3] 후보별 보조정보 계산...')
    sheets = pd.read_excel(INPUT_PATH, sheet_name=None, dtype={'종목코드': str})
    result = {}
    for name in SHEETS:
        if name not in sheets:
            continue
        print(f'  {name}: {len(sheets[name])}건')
        result[name] = enrich(sheets[name], start, end, regime)

    print('[3/3] 저장...')
    note = pd.DataFrame({'안내': [
        '이 표는 "사도 된다"는 신호가 아니라 후보를 사람이 고르기 쉽게 정리한 것입니다.',
        '백테스트(2021~, 전 종목, 거래비용 0.3% 반영)에서 이 신호의 평균 수익률은 마이너스였고, 랜덤 진입 대비 우위도 통계적으로 유의하지 않았습니다.',
        '그래서 최종 판단은 차트(지지/저항, 거래량), 섹터 분위기, 시장 상황을 직접 보고 하세요.',
        f'권장비중%는 손절 시 계좌 손실을 {RISK_PER_TRADE_PCT}%로 맞추는 계산이며 상한은 {MAX_WEIGHT_PCT}%입니다. 갭하락으로 손절가보다 더 크게 잃을 수
있습니다.',
        '손절폭%가 클수록 한 번 틀렸을 때 손실이 큽니다. 15% 이상이면 비중을 더 줄이는 쪽이 안전합니다.',
    ]})
    with pd.ExcelWriter(OUTPUT_PATH, engine='openpyxl') as writer:
        for name, d in result.items():
```

```
            d.drop(columns='봉', errors='ignore').to_excel(writer, sheet_name=name, index=False)
        if '보유종목' in sheets:
            sheets['보유종목'].to_excel(writer, sheet_name='보유종목', index=False)
        note.to_excel(writer, sheet_name='안내', index=False)
    print(f'완료: {OUTPUT_PATH}')


if __name__ == '__main__':
    main()
```